

第三章 RISC-V汇编及其指令系统

- RISC-V概述
- **RISC-V汇编语言**
- RISC-V指令表示
- 案例分析



第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**

- 汇编语言简介
- RISC-V汇编指令概览
- RISC-V常用汇编指令
- 过程调用及栈的使用
- 模拟软件介绍



汇编语言是什么？

- 汇编语言（**Assembly Language**）是一种“低级”语言，直接接触最底层的硬件，需要对底层硬件非常熟悉才能编写出高效的汇编程序。
- 在汇编语言中，用**助记符**（一些容易理解和记忆的字母、单词）**代替机器指令的操作码**，用**地址符号或标号代替指令或操作数的地址**。
- 例如：用ADD代表加减，MOV代表数据传递等等。

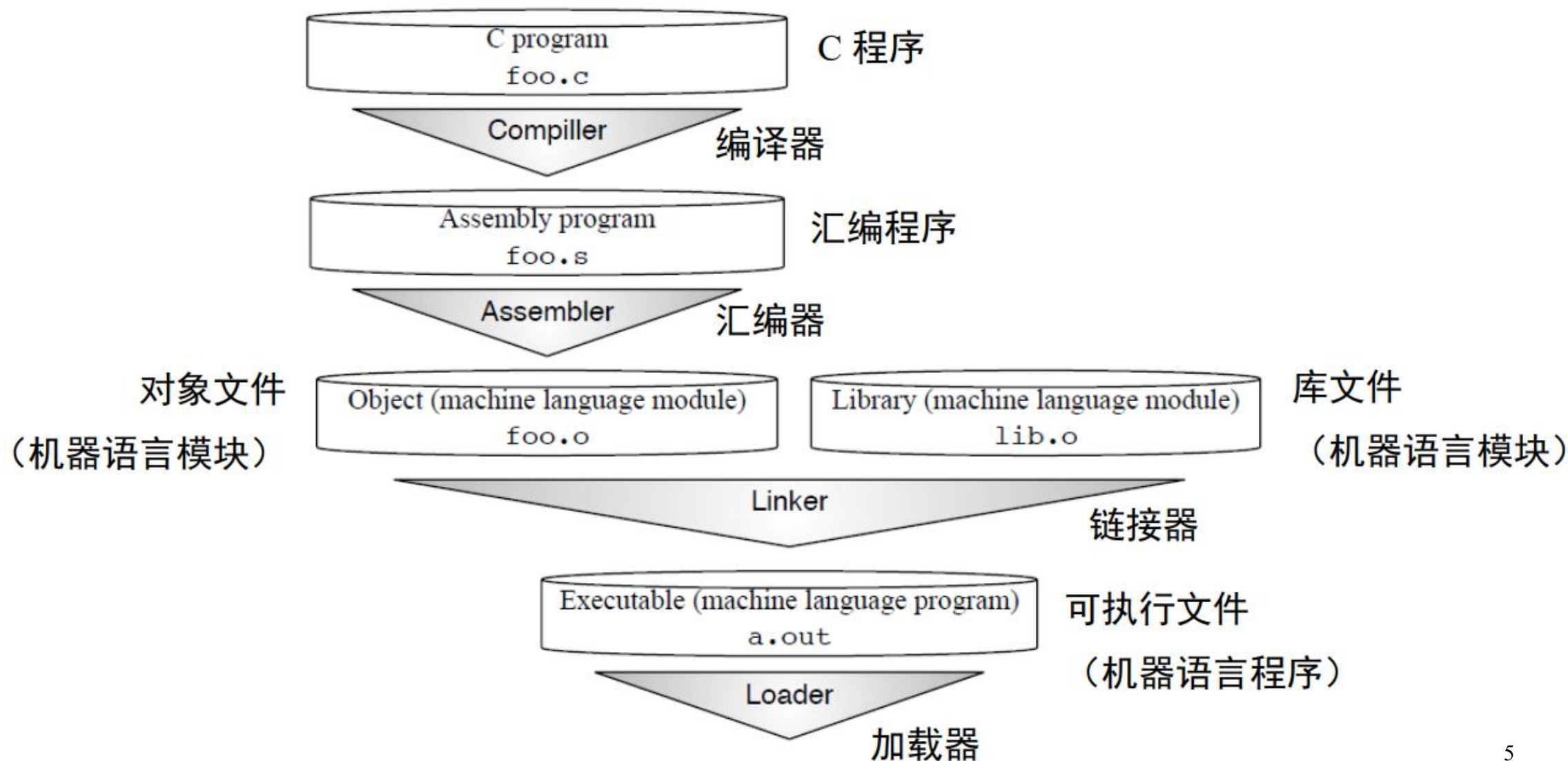
C语言程序—>机器语言程序

- C语言程序转换成 机器语言程序，大致需要以下四步：
 1. 将高级语言程序编译成汇编语言程序
 2. 用机器语言组装成目标模块
 3. 链接器将多个模块与库程序组合在一起以解析所有引用
 4. 加载器将机器代码放入适当的存储器位置以供处理器执行

注：不一定严格按照这四个步骤执行。

为了加快转换过程，可以跳过或将一些步骤组合到一起。例如：编译器直接生成目标模块，一些系统使用链接加载器执行最后两个步骤。

翻译并启动程序(UNIX)



编译器、汇编器

- **编译器** 将高级语言程序转换为机器能理解的符号形式——汇编语言程序（assembly language program）。
- **汇编器** 将汇编语言转换为目标文件，该目标文件是机器指令、数据和将指令正确放入内存所需信息的组合。
 - 汇编器还可以处理机器指令的常见变体，这类指令称为**伪指令**。

如：li x9, 123 汇编器将其转化为 addi x9, x0, 123

mv x11, x10 汇编器将其转化为 addi x11, x10, 0

and x9, x10, 15 汇编器将其转化为 andi x9, x10, 15

链接器、加载器、动态链接库

- **链接器** 链接器将所有独立汇编的机器语言程序“缝合”在一起。由于独立编译和汇编每个过程，因此更改一行代码只需要编译和汇编一个过程，链接器有用的原因是修正代码要比重新编译和重新汇编快得多。
- **加载器** 将可执行程序放在主存中以准备执行的系统程序。
- **动态链接库(Dynamic Link Library)**是在执行期间链接到程序的库例程。DLL需要额外的空间来存储动态链接所需的信息，但不要求复制或链接整个库。在第一次调用例程时会付出大量的开销，但此后只需一个间接跳转。

汇编语言的优缺点

- 汇编语言的**缺点**：难读、难写、难移植
- 汇编语言的**优点**：灵活、强大
- 汇编语言的**应用场景**：
 - 需要直接访问底层硬件
 - 需要对性能进行优化

第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**
 - 汇编语言简介
 - **RISC-V汇编指令概览**
 - RISC-V常用汇编指令
 - 过程调用及栈的使用
 - 模拟软件介绍

RISC-V 汇编指令格式

- 一种通用指令格式

op **dst**, **src1**, **src2**

- 1个操作码，3个操作数
- op (**operation**)：操作的助记符
- dst (**destination**)：目标寄存器
- src1 (**source1**)：第一个源操作数寄存器
- src2 (**source2**)：第二个源操作数寄存器
- 通过一些限制来保持硬件简单

RISC-V 汇编格式

- 每一行最多一条指令
- 每一条指令只有一个操作
- 汇编语言可以实现C语言中类似操作（= + - * / & | 等）
 - C语言中的操作会被分解为一条或者多条汇编指令
 - C语言中的一行会被编译为多行RISC-V汇编程序

RISC-V汇编指令操作对象

- 寄存器:

- 32个通用寄存器, $x0 \sim x31$ (注意: 仅涉及 RV32I 的通用寄存器组)
- 在 RISC-V 中, 算术逻辑运算所操作的数据必须直接来自寄存器
- $x0$ 是一个特殊的寄存器, 只用于全零
- 每一个寄存器都有别名便于使用, 实际硬件并没有任何区别
- RV32I指令集通用寄存器是32位的; RV64I指令集的通用寄存器是64位

- 内存:

- 可以执行在寄存器和内存之间的数据读写操作
- 读写操作使用字节 (Byte) 为基本单位进行寻址
- RV64可以访问最多 2^{64} 个字节的内存空间, 即 2^{61} 个存储单元

汇编语言的变量---寄存器

- 汇编语言不能使用变量（C、JAVA可以）
 - `int a; float b;`
 - 寄存器变量没有数据类型
- 汇编语言的操作对象以寄存器为主
 - 好处：寄存器是最快的数据存储单元
 - 缺陷：寄存器数量有限，需要充分和高效地使用各寄存器

32个RISC-V寄存器

寄存器	助记符	备注
x0	zero	固定值为0
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针(Global Pointer)
x4	tp	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器 (temporary register)
x8	s0/fp	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器 (Save Register)
x10-x11	a0-a1	过程参数(function argument)/过程返回值(Return Value)
x12-x17	a2-a7	过程参数(function argument)
x18-x27	s2-s11	save寄存器
x28-x31	t3-t6	临时寄存器

RISC-V汇编语言指令分类

- 算术运算: add、sub、mul、div、add**i** 后缀**i** —> **immediate**
- 逻辑运算: and、or、xor、and**i**、ori、xori
- 移位操作: sll、srl、sra、sll**i**、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lh**u**、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、slt**u**、slt**i****u** 后缀**u** —> **unsigned**
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

伪指令: 方便程序员使用的指令，是以上指令的变化或组合。

RISC-V 汇编语言

类别	指令	示例	含义	注释
算术运算	加	add x5, x6, x7	$x5 = x6 + x7$	三寄存器操作数; 加
	减	sub x5, x6, x7	$x5 = x6 - x7$	三寄存器操作数; 减
	立即数加	addi x5, x6, 20	$x5 = x6 + 20$	用于加常数
数据传输	取双字	ld x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取双字到寄存器
	存双字	sd x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	从寄存器存双字到存储器
	取字	lw x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取字到寄存器
	取字 (无符号数)	lwu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取无符号字到寄存器
	存字	sw x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	从寄存器存字到存储器
	取半字	lh x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取半字到寄存器
	取半字 (无符号数)	lhu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取无符号半字到寄存器
数据传输	存半字	sh x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	从寄存器存半字到存储器
	取字节	lb x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取字节到寄存器
	取字节 (无符号数)	lbu x5, 40(x6)	$x5 = \text{Memory}[x6 + 40]$	从存储器取无符号字节到寄存器
	存字节	sb x5, 40(x6)	$\text{Memory}[x6 + 40] = x5$	从寄存器存字节到存储器
	取保留字	lr.d x5, (x6)	$x5 = \text{Memory}[x6]$	取; 原子交换的前半部分
	存条件字	sc.d x7, x5, (x6)	$\text{Memory}[x6] = x5; x7 = 0/1$	存; 原子交换的后半部分
	取立即数高位	lui x5, 0x12345	$x5 = 0x12345000$	取左移12位后的20位立即数
逻辑运算	与	and x5, x6, x7	$x5 = x6 \& x7$	三寄存器操作数; 按位与
	或	or x5, x6, x8	$x5 = x6 x8$	三寄存器操作数; 按位或
	异或	xor x5, x6, x9	$x5 = x6 \wedge x9$	三寄存器操作数; 按位异或
	与立即数	andi x5, x6, 20	$x5 = x6 \& 20$	寄存器与常数按位与
	或立即数	ori x5, x6, 20	$x5 = x6 20$	寄存器与常数按位或
	异或立即数	xori x5, x6, 20	$x5 = x6 \wedge 20$	寄存器与常数按位异或

移位操作	逻辑左移	sll x5, x6, x7	x5 = x6 << x7	按寄存器给定位数左移
	逻辑右移	srl x5, x6, x7	x5 = x6 >> x7	按寄存器给定位数右移
	算术右移	sra x5, x6, x7	x5 = x6 >> x7	按寄存器给定位数算术右移
	逻辑左移立即数	slli x5, x6, 3	x5 = x6 << 3	根据立即数给定位数左移
	逻辑右移立即数	srlr x5, x6, 3	x5 = x6 >> 3	根据立即数给定位数右移
	算术右移立即数	srair x5, x6, 3	x5 = x6 >> 3	根据立即数给定位数算术右移
条件分支	相等即跳转	beq x5, x6, 100	if (x5 == x6) go to PC+100	若寄存器数值相等则跳转到PC相对地址
	不等即跳转	bne x5, x6, 100	if (x5 != x6) go to PC+100	若寄存器数值不等则跳转到PC相对地址
	小于即跳转	blt x5, x6, 100	if (x5 < x6) go to PC+100	若寄存器数值比较结果小于则跳转到PC相对地址
	大于等于即跳转	bge x5, x6, 100	if (x5 >= x6) go to PC+100	若寄存器数值比较结果大于或等于则跳转到PC相对地址
	小于即跳转（无符号）	bltu x5, x6, 100	if (x5 < x6) go to PC+100	若寄存器数值比较结果小于则跳转到PC相对地址（无符号）
	大于等于即跳转（无符号）	bgeu x5, x6, 100	if (x5 >= x6) go to PC+100	若寄存器数值比较结果大于或等于则跳转到PC相对地址（无符号）
无条件跳转	跳转-链接	jal x1, 100	x1 = PC+4; go to PC+100	用于PC相关的过程调用
	跳转-链接（寄存器地址）	jalr x1, 100(x5)	x1 = PC+4; go to x5+100	用于过程返回；非直接调用

第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**
 - 汇编语言简介
 - RISC-V汇编指令概览
 - **RISC-V常用汇编指令**
 - 过程调用及栈的使用
 - 模拟软件介绍

常用汇编语言指令

- **算术运算: add、sub、mul、div、addi**
- 逻辑运算: and、or、xor、andi、ori、xori
- 移位操作: sll、srl、sra、slli、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、sltu、sltiu
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

算术运算指令——(整数)加减法

- **加法: add**

- 语法: `add rd, rs1, rs2` (rd, rs1, rs2 为x0-x31中的某一个寄存器)
- 功能: 把寄存器rs2的**值**与寄存器 rs1的**值**相加, **结果写入寄存器rd (忽略算术溢出)**, 记为 **$rd=rs1+rs2$** 。

- **减法: sub**

- 语法: `sub rd, rs1, rs2`
- 功能: **$rd=rs1 - rs2$ (忽略算术溢出)**

- 溢出是因为计算机中表达数本身是**有范围限制**的

- 计算的结果没有足够多的位数进行表达
- RISC-V 忽略溢出问题, 高位被截断, 低位写入目标寄存器

算术运算指令——整数加减法的例子

- 假设：变量a，b和c被分别放置在寄存器x1, x2,和x3中
- 整数的加法（**add**）
 - C语言: $a = b + c$
 - RISC-V: `add x1, x2, x3`
- 整数的减法（**sub**）
 - C语言: $a = b - c$
 - RISC-V: `sub x1, x2, x3`

算术运算举例

- 假设 $a \rightarrow x1$, $b \rightarrow x2$, $c \rightarrow x3$, $d \rightarrow x4$, $e \rightarrow x5$, 将下面一段C语言程序编译成RISC-V汇编指令:

$a = (b + c) - (d + e);$

add x6, x4, x5

tmp1 = d + e

add x7, x2, x3

tmp2 = b + c

sub x1, x7, x6

a = tmp2-tmp1=(b + c) - (d + e)

- 指令中可以看到如何使用临时寄存器 (x5-x7的助记符t0-t2)
 - 一个简单的C语言表达式可能需要多条汇编语句
- # 符号后面是注释

RISC-V (整数)乘法与除法指令

- **乘法: mul**

- 使用语法: `mul rd, rs1, rs2`
- 功能描述: $rd = rs1 \times rs2$ 寄存器rs2的值乘以寄存器rs1的值, 结果写入寄存器rd, **忽略算术溢出**。

- **除法: div**

- 使用语法: `div rd, rs1, rs2`
- 功能描述: $rd = rs1 \div rs2$ 寄存器rs1的值除以寄存器rs2的值, 向零舍入, 将这些数视为**二进制补码**, 商写入寄存器rd。

- **取余数: rem**

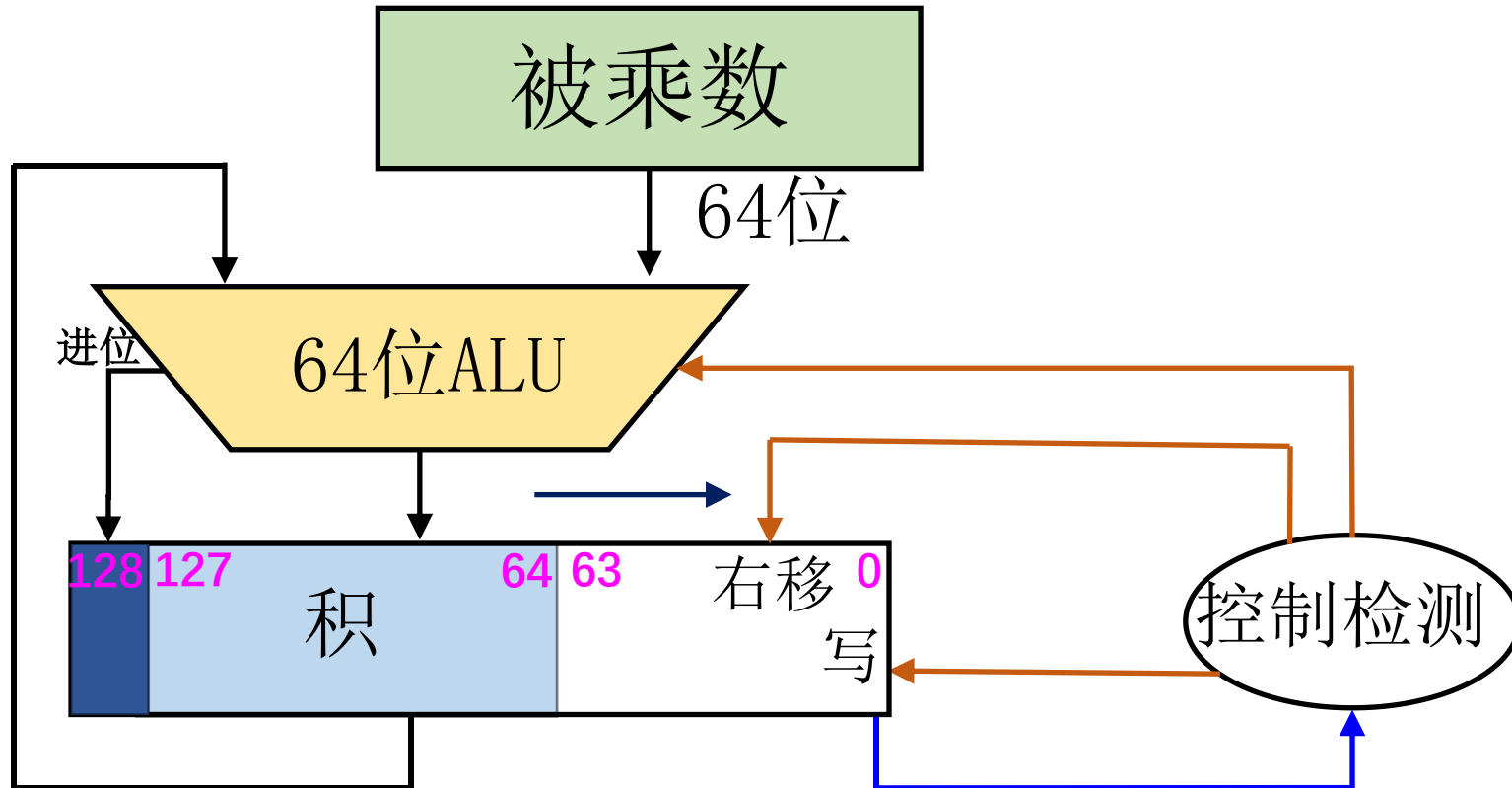
- 使用语法: `rem rd, rs1, rs2`

RISC-V 乘法指令

- 积的长度是乘数和被乘数长度之和。两个32位数的积是64位。
- 为了得到有符号或无符号的64位积，RISC-V中有四个指令：
 - 要得到整数32位乘积（64位中的低32位）就用mul指令；
 - 要得到高32位积，如果操作数都是有符号数，就用mulh指令；
如果操作数都是无符号数，就用mulhu指令；
如果一个有符号一个无符号，可以用mulhsu指令。

31	25	24	20	19	15	14	12	11	7	6	0	
0000001		rs2		rs1		000		rd		0110011		R mul
0000001		rs2		rs1		001		rd		0110011		R mulh
0000001		rs2		rs1		010		rd		0110011		R mulhsu
0000001		rs2		rs1		011		rd		0110011		R mulhu

乘法器硬件示意图



- A. 积寄存器右64位：乘数
 - B. 若乘数最右端为**1**：将积寄存器[127:64]位取出与被乘数进行**加法**运算，结果写入积寄存器
 - C. 积寄存器整体右移一位
- B和C循环64次
- 积寄存器中即为最终结果

RISC-V 乘法指令

- 如果需要获得完整的64位积，建议的指令序列为

`mulh[[s]u] rdh, rs1, rs2`

`mul rdl, rs1, rs2`

- 源寄存器必须使用**相同的顺序**，rdh要与rs1和rs2都**不相同**。
这样底层的微体系结构就会把两条指令合并成**一次**乘法操作，而不是两次乘法操作

除法指令举例

分别求x6/x7的商和余数。

div x5, x6, x7 # x5 = x6/x7

rem x8, x6, x7 # x8 = x6 mod x7

0000001	rs2	rs1	100	rd	0110011	R div
0000001	rs2	rs1	101	rd	0110011	R divu
0000001	rs2	rs1	110	rd	0110011	R rem
0000001	rs2	rs1	111	rd	0110011	R remu

RISC-V 中的立即数

- 数值常数被称为立即数 (immediates)
- 立即数指令的语法: **opi dst, src, imm**
 - 操作码的最后一个字符为i的, 会将第二个操作数认为是一个立即数 (经常用后缀来指明操作数的类型, 例如无符号数unsigned的后缀为u)
- 指令举例
 - addi rd, rs1, immediate** # rd=rs1 + **s**ext(immediate)
立即数表示范围有限 # 符号**扩展**立即数
 - addi x1, x2, 5** # x1=x2+5
 - addi x3, x3, 1** # x3=x3+1 (自增1)

特殊的寄存器x0 (zero)

- 0在程序中很常见，拥有一个自己的寄存器
- x0是一个特殊的寄存器，值恒为0，且不能被改变
 - **注意**: 在任意指令中，如果使用x0作为目标寄存器，将没有任何效果，仍然保持0不变
- 使用实例

add x3, x0, x0 # x3=0

add x1, x2, x0 # x1=x2

add x0, x0, x0 # nop

RISC-V 中的立即数

$f = g - 10$ (in C)

`addi x3, x4, -10` (in RISC-V)

问题：在RISC-V中有`sub`，没有立即数减法，为什么？

- 如果一个操作可以分解成一个更简单的操作，那就不要包含它。



常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- **逻辑运算: and、or、xor、andi、ori、xori**
- 移位操作: sll、srl、sra、slli、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、sltu、sltiu
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

逻辑运算指令

语法	功能	描述
and rd, rs1, rs2	$rd = rs1 \ \& \ rs2$	按位“与”操作，结果写入寄存器rd
andi rd, rs1, imm	$rd = rs1 \ \& \ sext(imm)$	把符号位扩展的立即数和寄存器rs1的值进行按位与，结果写入寄存器rd
or rd, rs1, rs2	$rd = rs1 \ \ rs2$	按位“或”操作，结果写入寄存器rd
ori rd, rs1, imm	$rd = rs1 \ \ sext(imm)$	把符号位扩展的立即数和寄存器rs1的值进行按位或，结果写入寄存器rd
xor rd, rs1, rs2	$rd = rs1 \ \wedge \ rs2$	按位“异或”操作，结果写入寄存器rd
xori rd, rs1, imm	$rd = rs1 \ \wedge \ sext(imm)$	把符号位扩展的立即数与寄存器rs1的值按位“异或”，结果写入寄存器rd

位操作指令实例

假设: $a \rightarrow x1$, $b \rightarrow x2$, $c \rightarrow x3$

指令	C	RSC-V
And	$a = b \ \& \ c;$	and x1, x2, x3
And Immediate	$a = b \ \& \ 8;$	andi x1, x2, 8
Or	$a = b \ \ c;$	or x1, x2, x3
Or Immediate	$a = b \ \ 22;$	ori x1, x2, 22
eXclusive OR	$a = b \ ^ \ c;$	xor x1, x2, x3
eXclusive OR Immediate	$a = b \ ^ \ -5;$	xori x1, x2, -5

位操作举例

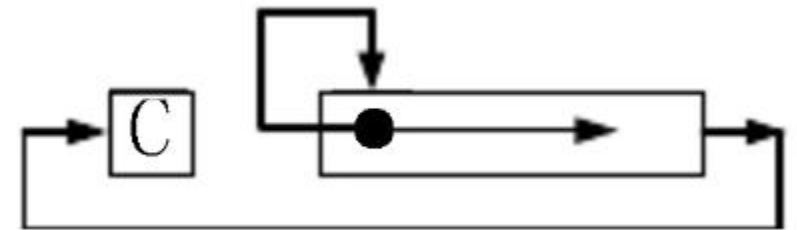
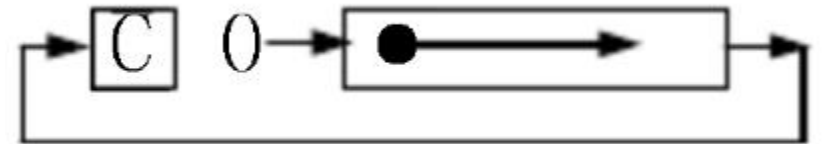
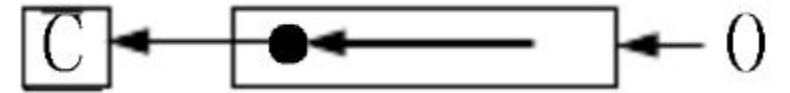
- 按位逻辑运算
 - 寄存器操作数: `and x5, x6, x7` # $x5 = x6 \& x7$
 - 立即数操作数: `andi x5, x6, 3` # $x5 = x6 \& 3$
- RISC-V中**没有NOT、按位取反**
 - 一个二进制数 `xor 111111112 → ?`
 - 例: `xori x5, x6, -1`
$x5 = \overline{x6}$

常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- 逻辑运算: and、or、xor、andi、ori、xori
- **移位操作: sll、srl、sra、slli、srli、srai**
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、sltu、sltiu
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

移位运算

- 左移相当于乘以2
 - 左移右边补0
 - 左移操作比乘法更快
- 逻辑右移：在最高位添加0
- 算术右移：在最高位添加符号位
- **移位的位数**可以是立即数或者寄存器中的值



移位指令

- slli, srli, srai只需要最多移动**63位** (对于RV**64**), 只会使用immediate低6位的值 (I型指令) 

imm[11:0]		rs1	111	rd	0010011	andi
000000	shamt	rs1	001	rd	0010011	slli

Instruction Name	RISC-V
Shift Left Logical	sll rd, rs1, rs2
Shift Left Logical I mmediate	slli rd, rs1, shamt
Shift Right Logical	srl rd, rs1, rs2
Shift Right Logical Immediate	srli rd, rs1, shamt
Shift Right A rithmetic	sr a rd, rs1, rs2
Shift Right A rithmetic Immediate	sr a i rd, rs1, shamt

移位运算举例

- 逻辑移位指令 `srl, srli, sl, slli`

`slli x11, x12, 2` `# x11=x12<<2`

- 算术移位指令 `sra, srai`

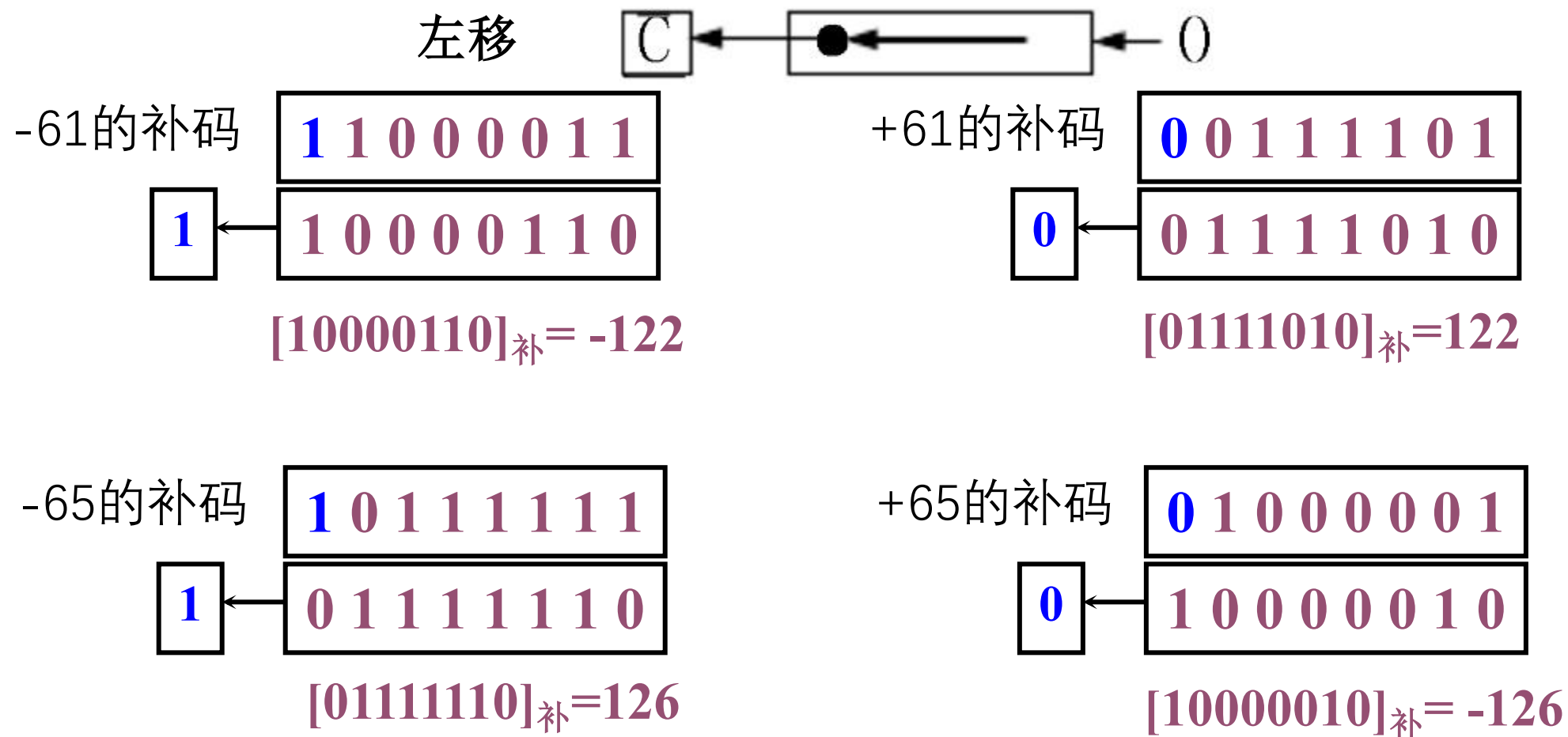
`srai x10, x10, 4` `# x10=x10>>4`

- 为什么没有算术左移指令？

例：8位二进制数的补码移位

真值	补码	逻辑左移	算术左移	结果	逻辑右移1位	算术右移1位
127	01111111	11111110	01111110	错误	00111111	同逻辑右移
...				错误		
64	01000000	10000000	00000000	错误	00100000	同逻辑右移
63	00111111	01111110	01111110	正确	00011111	同逻辑右移
...				正确		
1	00000001	00000010	00000010	正确	00000000	同逻辑右移
-1	11111111	11111110	11111110	正确	01111111	11111111
...				正确	0xx.....x	11xx.....x
-64	11000000	10000000	10000000	正确	01100000	11100000
-65	10111111	01111110	11111110	错误	01011111	11011111
-66	10111110	01111100	11111100	错误	01011111	11011111
-128	10000000	00000000	10000000	错误	01000000	11000000

带进位的移位



左移后，“进位”与“首位”一致和不一致？

计算器

程序员

-61

0 - 61 =
-61

HEX C3
DEC -61
OCT 303
BIN 1100 0011

BYTE MS

按位 位移

计算器

程序员

HEX 86
DEC -122
OCT 206
BIN 1000 0110

BYTE MS

按位 位移

算术移位

逻辑移位

旋转循环移位

带进位旋转循

>> CE <>

) % ÷

8 9 ×

5 6 -

1 2 3 +

F +/- 0 . =

计算器

程序员

**-61 Lsh 1 =
-122**

HEX 86
DEC -122
OCT 206
BIN 1000 0110

BYTE MS

按位 位移

算术移位

逻辑移位

旋转循环移位

带进位旋转循

>> CE <>

) % ÷

8 9 ×

5 6 -

1 2 3 +

F +/- 0 . =

常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- 逻辑运算: and、or、xor、andi、ori、xori
- 移位操作: sll、srl、sra、slli、srli、srai
- **数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb等**
- 比较指令: slt、slti、sltu、sltiu
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

数据传输（主存访问指令）

- C语言中的变量会映射到寄存器中；很多数据结构（例如数组等），会映射到内存（主存）中
- 主存可以看作一维数组
- 数据传输指令**专门**在寄存器和主存之间传输数据
 - **Store**指令：数据从**寄存器** 到 **内存**
 - **Load**指令：数据从**内存** 到 **寄存器**
- 除数据传输指令外的RISC指令 **只** 在寄存器中操作

数据传输指令的格式

memop reg, offset(bAddrReg)

- memop: 操作的名字（load或者store的各种**变形**）
- reg: 源寄存器或者目标寄存器
- bAddrReg: 保存数据地址的基址寄存器 (base address register)
- offset: 地址偏移量（**字节**寻址），为立即数 (immediate)
- 访问的主存地址为 $\text{bAddrReg地址寄存器中的数值} + \text{offset}$
- 必须指向一个合法的地址

内存的字节寻址方式

- 现代计算机中的内存操作 以 字节（8bits）为基本单位
 - 不同的体系结构对字（word）的定义不同(RISC V中 1word = 4 bytes)
 - 内存是按照字节进行编址，不是按照字进行编址的
- RISC V中字地址之间有4个字节的距离
 - 字的地址为字内最低位字节的地址(**小端模式**)
 - 按字**对齐**，地址最后**两位为0**（4的倍数）
- C语言会自动按照数据类型来计算地址
- 在汇编中**需要程序员自己计算**

地址					地址
1111	高字节			低字节	11 00
1011	高字节			低字节	10 00
0111	高字节			低字节	01 00
0011	高字节			低字节	00 00

内存与变量的大小 (variable size)

- 数据传输指令

lw reg, offset(bAddrReg) #从存储器 读取 字(word)到寄存器

sw reg, offset(bAddrReg) #从寄存器 存入 字(word)到存储器

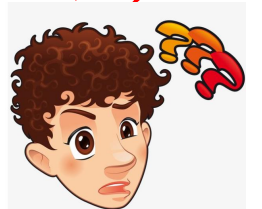
注意: bAddrReg+offset 按照字进行对齐, 即4的倍数

如何传输双字(8字节)、short型数据(2字节)或字符(1字节)?

双字: ld/sd

半字: lh/sh

字符: lb/sb。



主存数据访问指令：字节/半字/字操作

- 从主存地址 $rs1+offset$ 读不同字长数据，**符号扩展为64位**存入 rd
 - $lb\ rd, offset(rs1)$ $\#load\ byte$ (对于RV32则扩展为32位)
 - $lh\ rd, offset(rs1)$ $\#load\ halfword$
 - $lw\ rd, offset(rs1)$ $\#load\ word$
- 从主存读一个**无符号**的字节/半字/字：**零扩展为64位**存入 rd
 - $lbu\ rd, offset(rs1)$ $\#load\ byte\ unsigned$
 - $lhu\ rd, offset(rs1)$
 - $lwu\ rd, offset(rs1)$
- 将一个字节/半字/字写入主存 $rs1+offset$ （存**最右边8/16/32位**)
 - $sb\ rs2, offset(rs1)$
 - $sh\ rs2, offset(rs1)$
 - $sw\ rs2, offset(rs1)$

数据传输指令示例

请用RISC-V汇编语言（RV32）实现以下C语言代码的功能。

```
int A[100];
```

(in C)

```
A[10] = A[3] + a;
```

(RISC-V中一个字是32位，int类型是32位)

假设数组A的首地址保存在x6中，a保存在x5中 (RISC-V)

lw	x10, 12(x6)	# x10 = A[3]
add	x11, x5, x10	# x11 = A[3]+a
sw	x11, 40(x6)	# A[10] = A[3]+a

数据传输指令举例(读内存指令)

`long long int A[100];` // (in C, **long long int**类型是64位)

`g = h + A[3];`

设 $A[] \rightarrow x15$, $h \rightarrow x12$, 用RISC V64汇编实现上述功能。

ld x10, **24(x15)** # (双字是64位, 8字节)

add x11, **x12**, x10 # $g = h + A[3]$

- 基址寻址

- A[3]的地址为基址寄存器(x15) 的内容+ 偏移量(24)

数据传输指令举例(写内存指令)

long long int A[100]; (in C)

A[10] = h + A[3];

假设A[] $\rightarrow$ x15, h $\rightarrow$ x12

ld x10, 24(x15) # x15为A[0]地址 (in RISC-V**64**)

add x10, x12, x10 # x10=h + A[3]

sd x10, **80(x15)** # A[**10**] = h + A[3]

- 基址寻址

- 基址寄存器(x15)的内容 + 偏移量(80)

传输一个字节数据

- 使用字类型指令，配合位的**掩码**来操作字节数据

```
lw    x11, 0(x1)
```

```
andi  x11, x11, 0xFF      # lowest byte
```

- 使用专门的字节传输指令

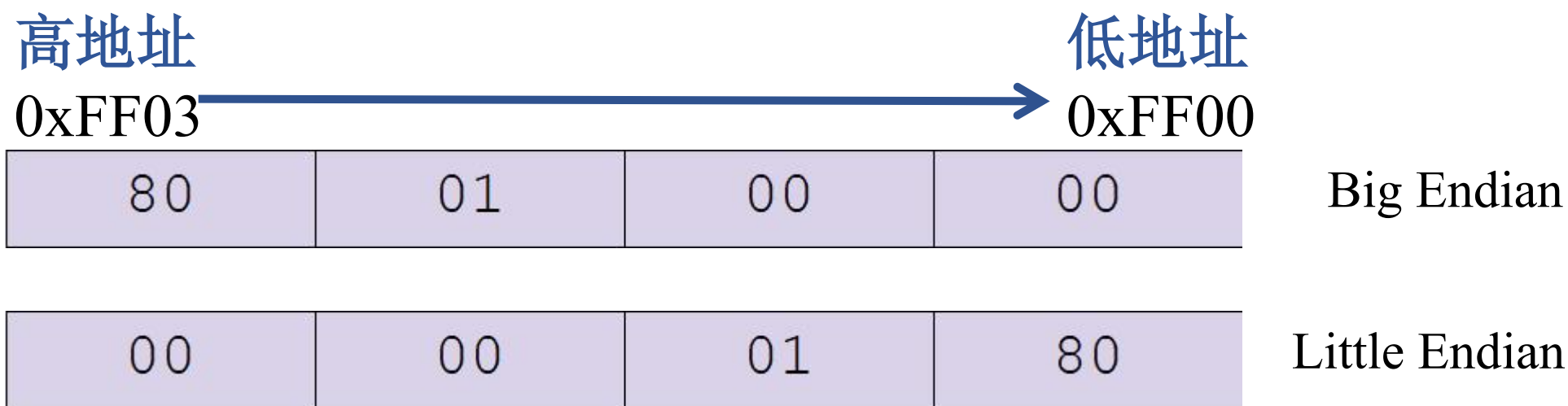
```
lb    x11, 0(x1)
```

```
sb    x11, 0(x1)
```

- 字节传输指令无需字对齐

字节排布

- **大端**：字地址等于**最高**字节的地址（字内高字节在最低的地址）
- **小端**：字地址等于**最低**字节的地址（字内低字节在最低的地址）
- 假设地址0xFF00储存数据为 0x00000180



RISC-V 是小端模式

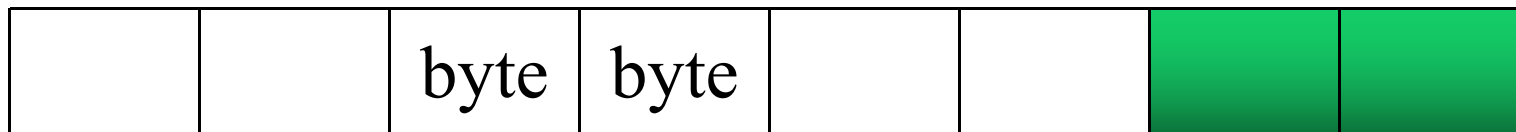
字节数据传输指令

- lb/sb使用的是**低字节**
- 如果是sb指令，高56位(RV64)/24位(RV32)**被忽略**
- 如果是lb指令，高56位(RV64)/24位(RV32)做**符号扩展**
- 设 mem[x1]中存储的数据为0x1020304050607080
(RV64) lb x12, **1**(x1) #x12=0x000000000000000070
 lb x12, **0**(x1) #x12=0xFFFFFFFFFFFFFFFF80
 sb x12, **2**(x1) #mem[x1]=0x1020304050807080

mem	10	20	30	40	50	60	70	80	x1
x12	FF	FF	FF	FF	FF	FF	FF	80	

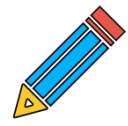
半字数据传输指令

- **lh** reg, off(bAddr) #load **half** word
- **sh** reg, off(bAddr) #store half word
- **lhu**、**shu** 无符号半字传输指令
 - off(bAddr)应该是2的倍数（对齐）
 - sh指令中高16位(RV32)/高48(RV64)**忽略**
 - lh指令中高16位(RV32)/高48(RV64)做**符号扩展**
 - **lbu**、**lhu**等带尾缀u的load类指令, 高位都做**0**扩展



主存数据访问指令：字节/半字/字操作

- 从地址为rs1的主存读字节/半字/字： **符号扩展为64位**存入rd
 - lb rd, offset(rs1) #load byte 对于**RV32**则扩展为**32**位
 - lh rd, offset(rs1) #load halfword
 - lw rd, offset(rs1) #load word
- 从主存读一个**无符号**的字节/半字/字： **零扩展为64位**存入rd
 - lbu rd, offset(rs1)
 - lhu rd, offset(rs1)
 - lwu rd, offset(rs1)
- 将一个字节/半字/字写入主存：保存**最右端的8/16/32位**
 - sb rs2, offset(rs1) # 保存**最右端的8位**（低8位）
 - sh rs2, offset(rs1) # 保存**最右端的16位**（低16位）
 - sw rs2, offset(rs1) # 保存**最右端的32位**（低32位）



常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- 逻辑运算: and、or、xor、andi、ori、xori
- 移位操作: sll、srl、sra、slli、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- **比较指令: slt、slti、sltu、sltiu**
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: jal、jalr

有符号数比较指令slt,slti

- **Set Less Than** (slt: 通过判断两个操作数的大小来对目标寄存器进行设置, **有符号比较**)
 - `slt dst, reg1, reg2`
 - 如果 (reg1中的值) < (reg2中的值), `dst = 1`, 否则`dst = 0`
- **Set Less Than Immediate** (slti)
 - `slti dst, reg1, imm`
 - 如果 (reg1中的值) < 立即数imm, `dst=1`, 否则`dst=0`

无符号数比较指令sltu,sltiu

- slt和slti的无符号版本 (按照无符号数比较):

sltu dst,src1,src2 #unsigned comparison

sltiu dst,src,imm # unsigned comparison against constant

```
addi    x10,x0,-1
```

```
slti     x11,x10,1
```

```
sltiu    x12,x10,1
```

无符号数比较指令sltu,sltiu

- slt和slti的无符号版本 (按照无符号数比较):

sltu dst,src1,src2 # unsigned comparison

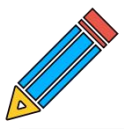
sltiu dst,src,imm #unsigned comparison against constant

(RISC-V32)

addi x10,x0,-1 #x10=0xFFFFFFFF

slti x11,x10,1 #x11=0x00000001(-1 < 1)

sltiu x12,x10,1 #x12=0x00000000($2^{32}-1 > 1$)



常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- 逻辑运算: and、or、xor、andi、ori、xori
- 移位操作: sll、srl、sra、slli、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、sltu、sltiu
- **条件分支: beq、bne、blt、bge、bltu、bgeu**
- 无条件跳转: jal、jalr

条件分支（跳转）指令

- C语言中有控制流
 - 比较语句/逻辑语句确定下一步执行的语句块
 - 可以用 { 和 } 定义语句块开始和结束
- RISC-V 汇编可通过标记（Label）定义语句块起始
 - 标记后面加一个冒号（例如 **Loop:** ）
 - 汇编的控制流就是跳转到标记的位置
 - 在C语言中有类似结构，但被认为是坏的编程风格（goto）

条件分支（跳转）指令

- 条件为真则转到标签所指的语句执行，否则顺序执行。
- `beq reg1, reg2, label` #branch if **e**qual
 - 如果reg1中的值=reg2中的值, 程序跳转到label处继续执行
- `bne reg1, reg2, label` #branch if **n**ot **e**qual
 - 如果reg1 **≠** reg2, 程序跳转到label处继续执行
- `blt reg1, reg2, label` #branch if **l**ess **t**han
 - 如果reg1 **<** reg2, 程序跳转到label处继续执行
- `bge reg1, reg2, label` #branch if **g**reater than or **e**qual
 - 如果reg1 **>=** reg2, 程序跳转到label处继续执行

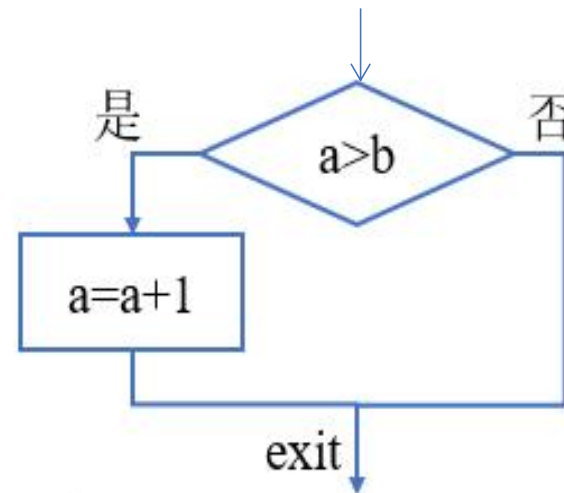
注意：**没有依据标志位**的跳转（与x86不同）

无符号版本命令有：**bltu**、**bgeu**

条件分支指令的例子

if (a > b) a += 1; // (C语言)

.....



假设：a保存在x22，b保存在x23

(RISC-V)

bge x23, x22, Exit

当b >= a时跳转

addi x22, x22, 1

a=a+1

Exit:

RISC-V 中的有符号与无符号

- 有符号和无符号在3个上下文环境中使用

–Signed vs. unsigned bit extension 符号扩展

lb, lh vs. lbu, lhu

–Signed vs. unsigned comparison 比较

slt, slti vs. sltu, sltiu

–Signed vs. unsigned branch 比较

blt, bge vs. bltu, bgeu

常用汇编语言指令

- 算术运算: add、sub、addi、mul、div
- 逻辑运算: and、or、xor、andi、ori、xori
- 移位操作: sll、srl、sra、slli、srli、srai
- 数据传输: ld、sd、lw、sw、lwu、lh、lhu、sh、lb、lbu、sb、lui
- 比较指令: slt、slti、sltu、sltiu
- 条件分支: beq、bne、blt、bge、bltu、bgeu
- 无条件跳转: **jal、jalr**

无条件跳转指令

- `jal rd, offset # (jump and link) #Program Counter`
 - 将下一条指令的地址**PC+4**保存在寄存器rd(一般用x1/ra)
 - 跳转到目标地址(PC+立即数偏移量, 有规定的转换规则)
 - 可用于过程**调用**
- `jalr rd, offset(rs1) #(jump and link register)`
 - 把PC+4存到rd中
 - 类似jal, 但跳转到rs1+offset地址处的指令 (**更远**)
 - 可以用于过程**返回**
- 如果rd用x0, 相当于只跳转不返回

32个RISC-V寄存器

寄存器	助记符	备注
x0	zero	固定值为0
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针(Global Pointer)
x4	tp	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器 (temporary register)
x8	s0/fp	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器 (Save Register)
x10-x11	a0-a1	过程参数 / 过程返回值(Return Value)
x12-x17	a2-a7	过程参数(Function Argument)
x18-x27	s2-s11	save寄存器
x28-x31	t3-6	临时寄存器

RISC-V汇编中的伪指令

- 方便程序员编程
- 通过汇编语言的变化或者组合来实现，不是硬件实现
- 例： **mv dst, src** #没有专门的硬件来实现

addi dst, src, 0

add dst, src, x0

j offset——>**jal x0,offset**

其它的伪指令

- **load i**mmEDIATE (li) 装入一个立即数
 - li dst, imm
 - 装入一个立即数到 dst寄存器
 - 被翻译为: addi dst,x0,imm
- **load a**ddress (la) 装入一个地址
 - la dst, label
 - 装入由Label指定的地址到 dst
- 指令手册 jr, neg....

其他的伪指令

PSEUDO INSTRUCTIONS

MNEMONIC	NAME	DESCRIPTION	USES
beqz	Branch = zero	if($R[rs1] == 0$) $PC = PC + \{imm, 1b'0\}$	beq
bnez	Branch $\neq$ zero	if($R[rs1] \neq 0$) $PC = PC + \{imm, 1b'0\}$	bne
fabs.s, fabs.d	Absolute Value	$F[rd] = (F[rs1] < 0) ? -F[rs1] : F[rs1]$	fsgnx
fmv.s, fmv.d	FP Move	$F[rd] = F[rs1]$	fsgnj
fneg.s, fneg.d	FP negate	$F[rd] = -F[rs1]$	fsgnjn
j	Jump	$PC = \{imm, 1b'0\}$	jal
jr	Jump register	$PC = R[rs1]$	jalr
la	Load address	$R[rd] = \text{address}$	auipc
li	Load imm	$R[rd] = imm$	addi
mv	Move	$R[rd] = R[rs1]$	addi
neg	Negate	$R[rd] = -R[rs1]$	sub
nop	No operation	$R[0] = R[0]$	addi
not	Not	$R[rd] = \sim R[rs1]$	xori
ret	Return	$PC = R[1]$	jalr
seqz	Set = zero	$R[rd] = (R[rs1] == 0) ? 1 : 0$	sltiu
snez	Set $\neq$ zero	$R[rd] = (R[rs1] \neq 0) ? 1 : 0$	sltu

伪指令实现

Pseudo	Real
<code>nop</code>	<code>addi x0, x0, 0</code>
<code>not rd, rs</code>	<code>xori rd, rs, -1</code>
<code>beqz rs, offset</code>	<code>beq rs, x0, offset</code>
<code>bgt rs, rt, offset</code>	<code>blt rt, rs, offset</code>
<code>j offset</code>	<code>jal x0, offset</code>
<code>ret</code>	<code>jalr x0, x1, offset</code>
<code>call offset</code> (if too big for just a <code>jal</code>)	<code>auipc x6, offset[31:12]</code> <code>jalr x1, x6, offset[11:0]</code>
<code>tail offset</code> (if too far for a <code>j</code>)	<code>auipc x6, offset[31:12]</code> <code>jalr x0, x6, offset[11:0]</code>

伪指令 vs. 硬件指令

- 硬件指令
 - 所有指令都是在**硬件**中直接**实现**了，可以直接执行。
- 伪指令
 - 方便汇编语言程序员使用的指令
 - 每一条伪指令会被翻译为1条或者多条硬件指令

Edit Execute

Text Segment

Program Arguments:

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x00300513	addi x10,x0,3	1: main: addi a0,zero,3 #想求fact(3),把立...
☐	0x00400004	0x00000317	auipc x6,0	2: <u>call fact</u> #结果存在x10中返回
☐	0x00400008	0x020300e7	jalr x1,x6,0x00000020	
☐	0x0040000c	0xffff03b7	lui x7,0xffffffff0	4: <u>li t2,0xffff000c</u>
☐	0x00400010	0x00c3839b	addiw x7,x7,12	
☐	0x00400014	0x04050313	addi x6,x10,0x00000040	5: addi t1,x10,0x40
☐	0x00400018	0x00638023	sb x6,0(x7)	6: sb t1,0(t2)

Labels

Label	Address ▲
fact.asm	
main	0x00400000
fact	0x00400024
L1	0x00400044

☒ Data ☒ Text

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x7fffffe0	0x00000000	0x00000000	0x00000000	0x00000003	0x00000000	0x0040000c	0x00000000	0x00000000
0x7ffff000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x7ffff020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x7ffff040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x7ffff060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x7ffff080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x7ffff0a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000



current sp



☒ Hexadecimal Addresses

☒ Hexadecimal Values

☐ ASCII

伪指令+条件分支转移指令示例

C语言

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

RISC V

$f \rightarrow x10, g \rightarrow x11, h \rightarrow x12, i \rightarrow x13, j \rightarrow x14$

```
bne x13, x14, Else
add x10, x11, x12
j Exit #伪指令
```

Else: sub x10, x11, x12

Exit:

用RV64I汇编实现C语言for循环结构

C语言

```
long long int A[20];  
long long int sum = 0;  
for (long long int i = 0; i < 20; i++)  
    sum += A[i];  
//假设数组A的首地址保存在 x8,  
//sum保存在x10中  
//i 保存在x11中
```



```
add x9, x8, x0    # x9=&A[0]  
add x10, x0, x0   # sum=0  
add x11, x0, x0   # i=0  
addi x13, x0, 20  # x13=20
```

Loop:

```
bge x11, x13, Done  
ld x12, 0(x9)     # x12=A[i]  
add x10, x10, x12  
addi x9, x9, 8    # &A[i+1]  
addi x11, x11, 1  # i++  
beq x0, x0, Loop
```

Done:

用RV64I汇编实现C语言while循环结构

C代码:

```
long long int save[100];
```

.....

```
while (save[i] == k) i += 1;
```

...

假设: i存储在x22中, k
存储在x24中, save数组
元素的地址保存在x25中。

RISC-V64I 代码:

Loop: slli x10, x22, 3 #8*i

add x10, x10, x25 #save[] + 8*i

ld x9, 0(x10) #save[i] → x9

bne x9, x24, Exit #save[i] != k Exit

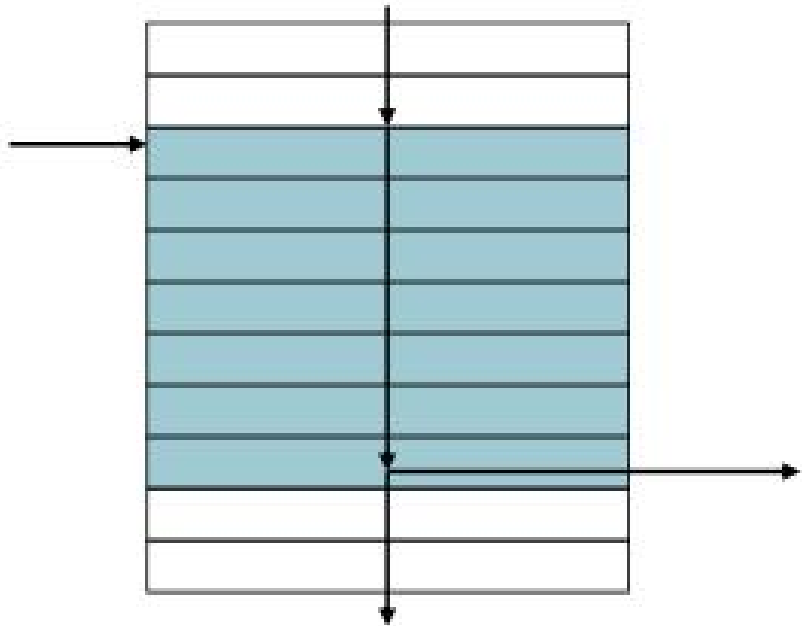
addi x22, x22, 1 #i = i + 1

beq x0, x0, Loop # 循环

Exit: ...

基本块

- 基本块是这样的指令序列
 - 没有嵌入分支（除非在末尾）
 - 没有分支目标（除非在开头）



- 编译器可以识别基本块以进行优化
- 先进的处理器能够加速基本块的执行

例：字符串复制

C代码：（以null字符结束的字符串）

```
void strcpy (char dest[], char src[]){  
    int i;  
    i = 0;  
    while ((dest[i]=src[i])!='\0')  
        i += 1;  
}
```

- 用x10保存数组dest地址， x11保存数组src地址， x19保存i

用x10保存数组dest地址， x11保存数组src地址， x19保存i

strcpy: addi sp, sp, -8

调整栈， 留出1个双字的空间

sd x19, 0(sp)

x19入栈

add x19, x0, x0

i=0 #i存在x19中

func1: add x5, x19, x11

x11为数组src首地址， x5 = src[i]的地址

lbu x6, 0(x5)

x6最低字节为src[i]

add x7, x19, x10

x10保存dest首地址， x7 = dest[i]的地址

sb x6, 0(x7)

dest[i] = src[i]

beq x6, x0, func2

若src[i] == '\0'， 则退出

addi x19, x19, 1

i = i + 1

jal x0, func1

下一次循环

func2: ld x19, 0(sp)

恢复x19原值

addi sp, sp, 8

从栈中弹出1个双字

jalr x0, 0(x1)

返回 #x1 (ra)



第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**

- 汇编语言简介
- RISC-V汇编指令概览
- RISC-V常用汇编指令
- **过程调用及栈的使用**
- 模拟软件介绍

过程调用的步骤

- 调用者caller vs. 被调用者callee
- (**Caller**)将参数放在过程可以访问的寄存器x10~x17/a0-a7
- (**Caller**)将控制权转交给过程(**Callee**)
- (**Callee**)获取过程所需的存储资源(包括保存现场)
- (**Callee**)执行过程中的操作
- (**Callee**)将结果值放在调用程序可以访问到的寄存器s0-s11
- (**Callee**)返回调用位置(x1/ra中的地址), 控制权交还调用者(**Caller**)

过程调用指令

- 过程调用:

jal x1, ProcedureAddress (跳转和链接)

- 将**下一条指令的地址**保存在x1/ra中
- 跳转到目标地址**ProcedureAddress**

- 过程返回:

jalr x0, 0(x1) (寄存器跳转和链接)

- 类似jal, 但跳转到 $0 + x1$ 中保存的地址
- 把x0用作目的寄存器 (实际x0不会被改变), 不会再返回了
- 同样可用于跳转到计算出的位置

过程调用相关的寄存器

- RISC-V实现过程调用的相关寄存器

- 返回地址寄存器 x1(ra)
- 参数寄存器 x10-x17(a0-a7)
- 返回值寄存器 x10-x11(a0-a1)
- 保存寄存器 x8,x9,x18-x27(s0,s1,s2-s11)
- 临时寄存器 x5-x7,x28-x31(t0-t2,t3-t6)
- 栈指针 x2(sp)

- C语言函数调用

```
int function(int a ,int b)      # a, b: s0, s1 调用者把参数存于s0,s1
{ return (a+b); }
```

RISC-V过程调用(人为设定返回地址)

```
1000 mv a0, s0
1004 mv a1, s1
1008 addi ra, zero, 1016 # ra = 1016
1012 j sum # jump to sum
1016 ... #跳转返回的下条指令
...
2000 sum: add a0, a0, a1
2004 jr ra # jalr x0,ra,0
jr: jump register #PC=rs1
```

- 返回为什么用**jr**而不用j?
 - 伪指令j跳转到给定的地址
 - **sum**可能在多处被调用，不能返回到一个固定地址。
 - 调用**sum**过程必须能够返回调用语句的下一句。

一条指令实现跳转和保存返回地址

• RISC-V过程调用

1000 mv a0, s0

1004 mv a1, s1

1008 addi ra, zero, 1016 # ra = 1016

1012 j sum # jump to sum

... # 下个指令

...

sum: add a0, a0, a1

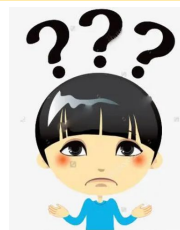
jr ra # jump register

• jal rd, offset # (jump and link)

• 将下一条指令的地址PC+4保存在寄存器rd(一般用x1)



为什么ra/x1 = 1012?



1008 jal ra, sum ra=?

RISC-V中过程调用要考虑的问题

- ◆除了过程嵌套之外，被调用过程中间不能用跳转指令跳出当前过程到另外一个过程然后再返回(有悖封装的本意)
- ◆高级语言函数体中一般使用局部变量
- ◆RISC-V汇编过程（子程序）使用寄存器（全局变量）
- ◆调用者(Caller)和被调用者(Callee)可能使用相同寄存器
 - 造成数据破坏
 - 被调用者需要保存可能被破坏的寄存器（现场）
 - 哪些寄存器属于现场？

RISC-V中调用过程时寄存器的使用

- 当被调用过程从执行中返回时，调用过程需要知道哪些寄存器可能已经更改，哪些寄存器保证不变。
- RISC-V过程调用时对需要保护现场的寄存器有如下**约定**
 - 保存 x1(ra), sp, **gp, tp**, x8-x9(s0-s1), x18-x27(s2-s11)
 - 不保存 x10-x17(a0-a7), x5-x7(t0-t2), x28-x31(t3-t6)
 - **详见（黑书P73图2-11）**

例：过程调用嵌套问题

```
main(){  
    ...;  
    numSquare(9,10);  
    ...; //4080  
}  
  
int numSquare (int x, int y){  
    x=mult(x,x);  
    ... //5000  
}
```

- main调用过程numSquare，寄存器ra保存返回地址4080
- 过程numSquare调用过程mult，寄存器ra需要保存返回地址5000，4080被覆盖
- 调用mult前需提前保存寄存器ra中的返回地址4080

栈的概念

- 客栈、货栈，临时“保存”人或者物的地方。
- 计算机中
 - 是一种数据结构（LIFO：Last In First Out后进先出）
 - 在调用函数时可以保存寄存器旧值（在主存中），在调用返回时取出并还原到寄存器中。
- 栈操作
 - 压栈：将数据放入栈
 - 弹栈：从栈中取出数据



运行Linux操作系统时RISC-V内存分配

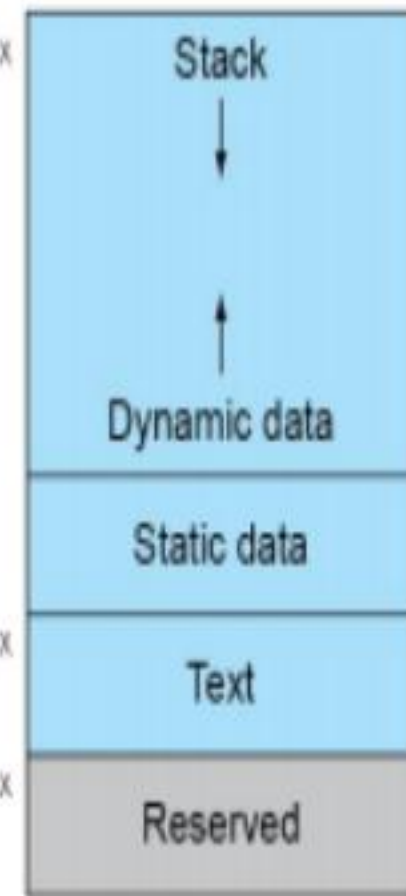
- TEXT 程序代码段
- Static Data 静态数据段：全局变量
 - 例如C中的静态变量、常数组和字符串
 - x3/gp(全局指针)初始化为适当的地址，加上正负偏移量可以访问这段内存空间
- Dynamic Data动态数据段：堆(heap)
 - 例如C中的malloc、Java中的new
- Stack 栈：自动存储
- PC、SP等的初值是**软件约定的**

SP → 0000 003f ffff fff0_{hex}

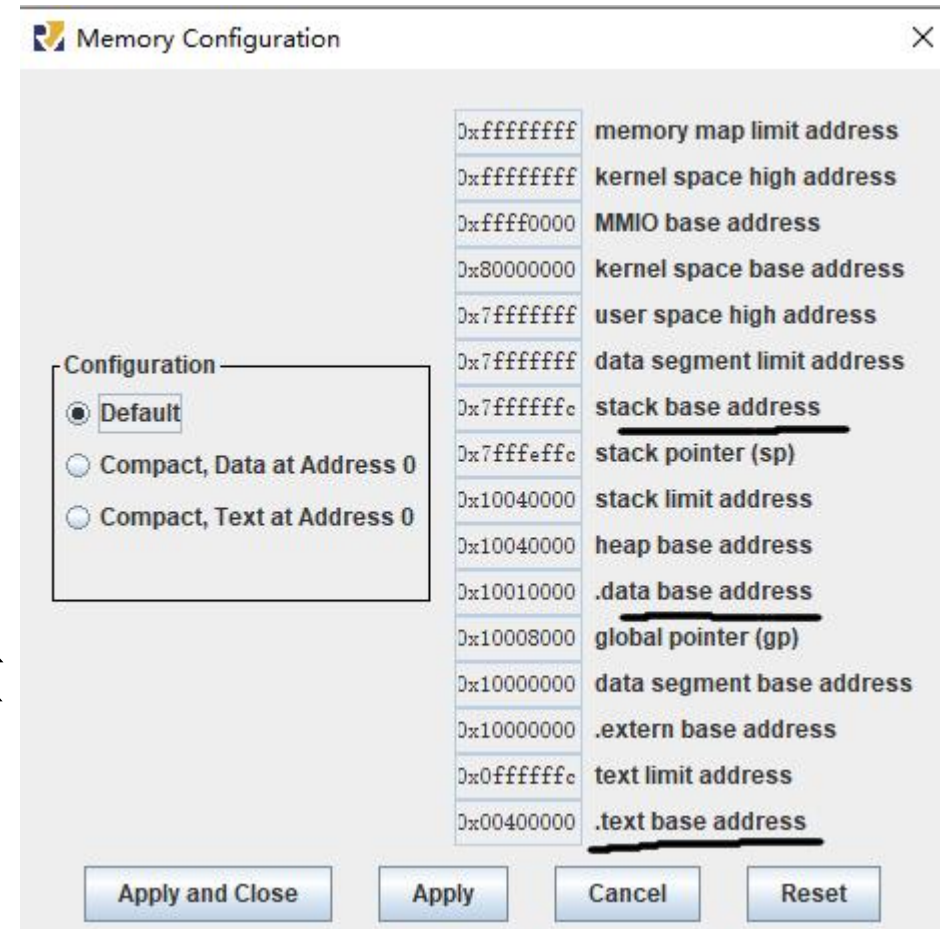
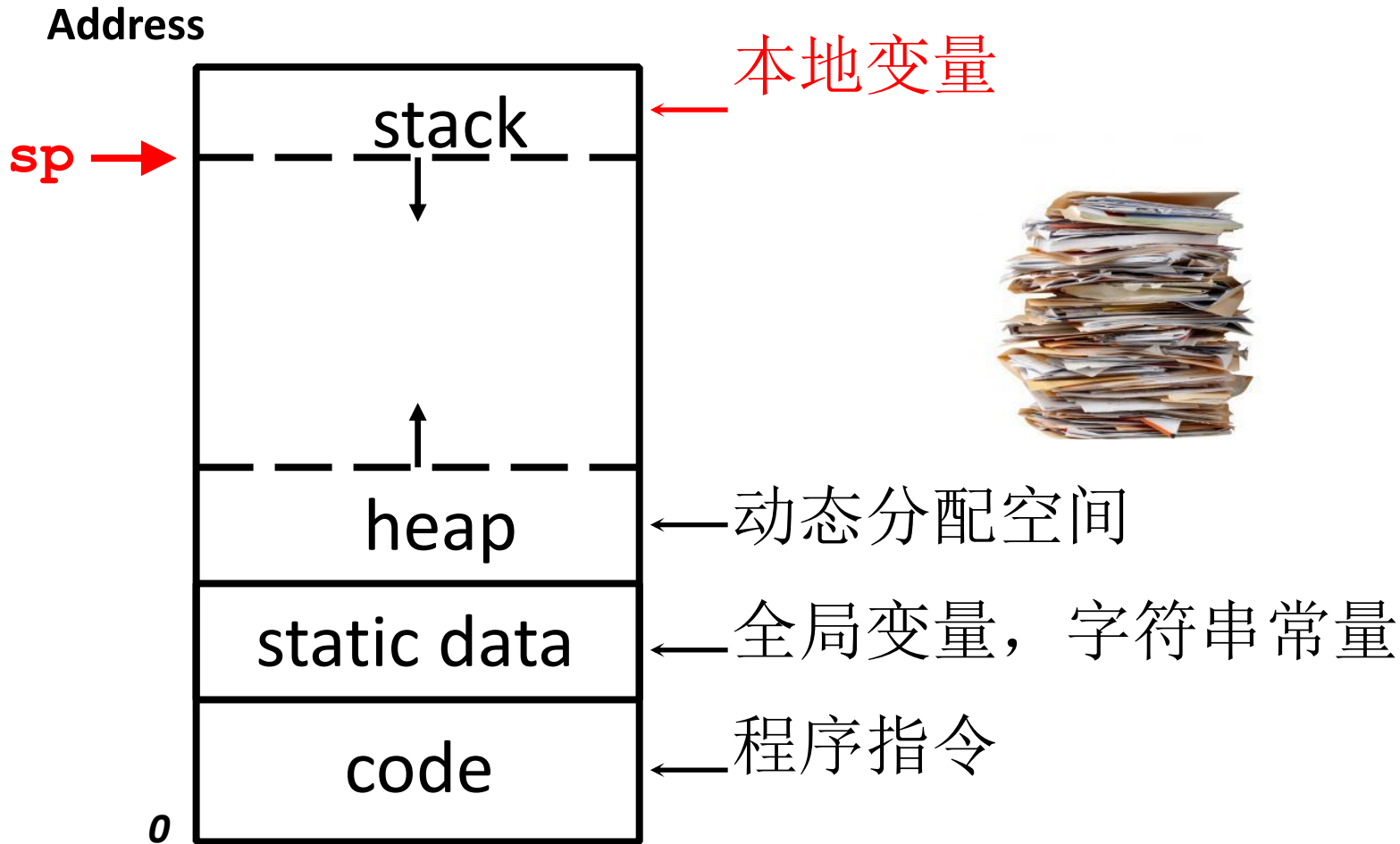
0000 0000 1000 0000_{hex}

PC → 0000 0000 0040 0000_{hex}

0

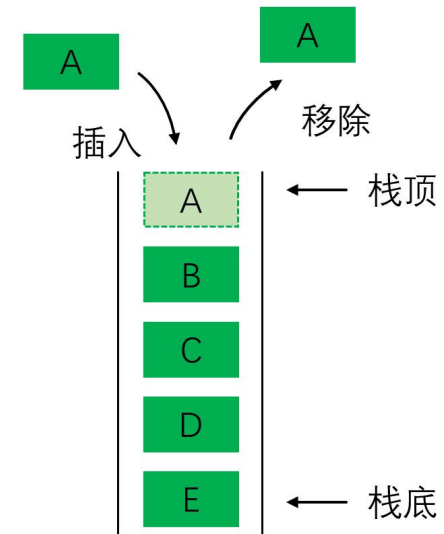


数据的内存排布情况



栈的使用

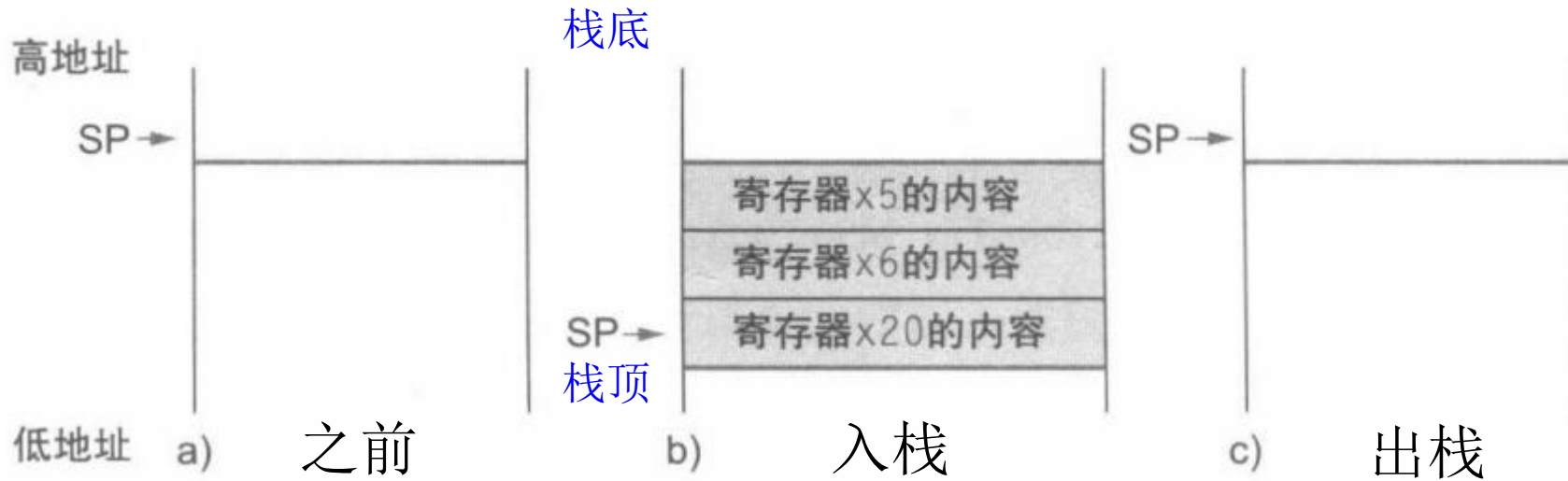
- x86指令集有入栈(压栈)指令push和出栈(弹栈)指令pop
- RISC V无专用的入栈和出栈指令
- sp(x2) 是RISC-V中的栈指针寄存器，用来保存栈顶地址
- 按惯例，RISC-V中栈按照从高到低的地址顺序增长（即栈底位于高地址，栈顶位于低地址）。
 - 压栈（入栈）向下移动栈指针(减 sp)
 - 弹栈（出栈）向上移动栈指针(增 sp)



栈的使用——续

- 栈指针寄存器sp总是指向栈中最后使用的空间
- 使用栈时，首先将栈指针减少所需的空间量，然后用需要保存的信息填充相应的存储空间。
- 例如，保存寄存器x9的内容到栈中(RV-32)。
 - `addi sp, sp, -4` # 将栈指针sp向低地址移动4个字节
 - `sw x9, 0(sp)` # 将寄存器x9中的内容压栈

过程调用时栈的变化



RISC-V64

压栈

```
addi sp, sp, -24
sd x5, 16(sp)
sd x6, 8(sp)
sd x20, 0(sp)
```

出栈

```
ld x20, 0(sp)
ld x6, 8(sp)
ld x5, 16(sp)
addi sp, sp, 24
```



```
int numSquare (int x, int y){
    return mult (x, x)+y; }
```

例：过程嵌套

- numSquare:

```
addi sp, sp, -8
```

λ 棧 sw ra, 4(sp)

sw al, 0(sp)

mv a1, a0

jal mult

$$1w\ a1, 0(sp)$$

```
add a0, a0, a1
```

出栈 lw ra, 4(sp)

```
addi sp, sp, 8
```

jr ra

mult:...

$$\#a0=x, \quad a1=y$$

预留栈空间

ra寄存器入栈

参数寄存器a1值y入栈

a1=a0=x, 为mult(x, x)准备参数

调用mult计算 $x*x$

出栈，恢复a1的值y

$$\# \mathbf{x} * \mathbf{x} + \mathbf{y}$$

返回地址出栈

恢复栈指针

例：用栈实现递归

```
C代码: long long int fact (long long int n){  
    if (n < 1) return 1;  
    else return n * fact(n - 1);  
}
```

- 参数n在x10中
- 结果在x10中

C (gcc 9.3, C17 + GNU extensions)
([known limitations](#))

```
1  int  factorial(int  x);
2  int  main()  {
→ 3      factorial(4);
4      return 0;
5  }
6  int  factorial(int  x)
7  {
8      if (x == 0 )
9      {
10         return 1;
11     }
12     else
13         return x * factorial(x-1);
14 }
```

[Edit this code](#)

→ line that just executed

→ next line to execute



<< First

< Prev

Next >

Last >>

Step 1 of 22

[Customize visualization](#)

Stack

Heap

main

RV汇编用栈实现递归求阶乘fact（黑书72页）

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd x1, 8(sp)      # 返回地址保存到栈中
      sd x10, 0(sp)     # 参数n保存到栈中
      addi x5, x10, -1
      bge x5, x0, L1    # 若参数n-1 >= 0, 则跳转到L1
      addi x10, x0, 1   # 若n-1 < 0, 返回值1并存入x10
      addi sp, sp, 16   # 释放2个双字栈空间, 不用恢复
      jalr x0, 0(x1)    # 返回fact的调用者
L1:   addi x10, x10, -1 # n = n - 1
      jal x1, fact      # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, x10, 0    # 从fact(0)开始, 结果存到t1
      ld x10, 0(sp)     # 恢复调用者的参数
      ld x1, 8(sp)      # 恢复调用者的返回地址
      addi sp, sp, 16   # 释放2个双字的栈空间
      mul x10, x10, t1  # 计算m * fact(m-1) (m从1开始)
      jalr x0, 0(x1)    # 返回calc
```

RV汇编用栈实现递归求阶乘fact(2)

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd ra, 8(sp)      # 返回地址保存到栈中
      sd a0, 0(sp)      # 参数n=2保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1  # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1  # 若n-1 < 0, 返回值1并存入a0
      addi sp, sp, 16   # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra)  # 返回fact(0)的调用者(执行Calc)
L1:   addi a0, a0, -1    # n = n - 1
      jal ra, fact      # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0     # 从fact(0)开始, 结果存到t1
      ld a0, 0(sp)      # 恢复调用者的参数
      ld ra, 8(sp)      # 恢复调用者的返回地址
      addi sp, sp, 16   # 释放2个双字的栈空间
      mul a0, a0, t1    # 计算m * fact(m-1) (m从1开始)
      jalr x0, 0(ra)    # 返回calc
```

SP →



STEP 1

注: $x_{1,i}$ 代表 x_1 寄存器存储的不同的返回地址值。

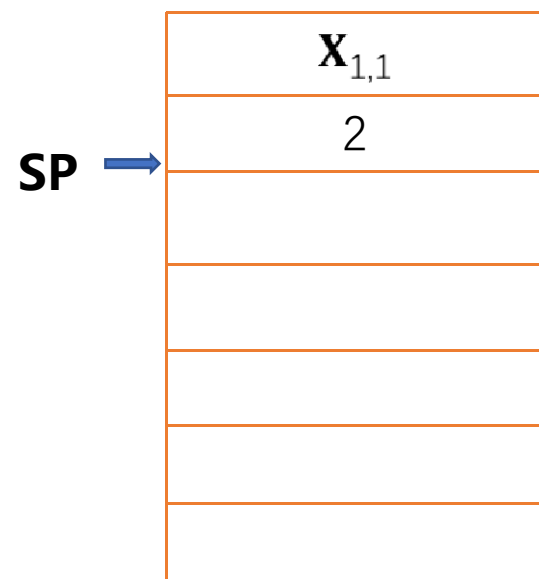
每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码;

蓝色代表改变栈后继续执行的代码。

fact(2)递归调用fact(1)

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd ra, 8(sp)      # 返回地址保存到栈中
      sd a0, 0(sp)      # 参数n=1保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1  # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1  # 若n-1 < 0, 返回值1并存入a0
      addi sp, sp, 16   # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra)  # 返回fact(0)的调用者(执行Calc)
L1:   addi a0, a0, -1    # n = n - 1
      jal ra, fact      # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0     # 从fact(0)开始, 结果存到t1
      ld a0, 0(sp)      # 恢复调用者的参数
      ld ra, 8(sp)      # 恢复调用者的返回地址
      addi sp, sp, 16   # 释放2个双字的栈空间
      mul a0, a0, t1    # 计算m * fact(m-1) (m从1开始)
      jalr x0, 0(ra)    # 返回calc
```



STEP 2

注: $x_{1,i}$ 代表 x_1 寄存器存储的不同的返回地址值。

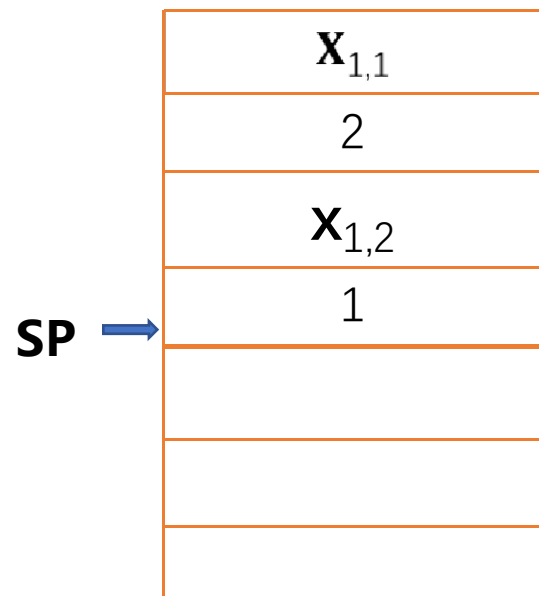
每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码;

蓝色代表改变栈后继续执行的代码。

fact(1)递归调用fact(0)

```
fact: addi sp, sp, -16  # 留出2个双字的栈空间
      sd ra, 8(sp)      # 返回地址保存到栈中
      sd a0, 0(sp)      # 参数n=0保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1  # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1  # 若n-1 < 0, 返回值1并存入a0
      addi sp, sp, 16    # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra)   # 返回fact(0)的调用者(执行Calc)
L1:   addi a0, a0, -1    # n = n - 1
      jal ra, fact      # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0    # 从fact(0)开始, 结果存到t1
      ld a0, 0(sp)      # 恢复调用者的参数
      ld ra, 8(sp)      # 恢复调用者的返回地址
      addi sp, sp, 16    # 释放2个双字的栈空间
      mul a0, a0, t1     # 计算1 * fact(1-1) (m从1开始)
      jalr x0, 0(ra)    # 返回calc
```



STEP 3

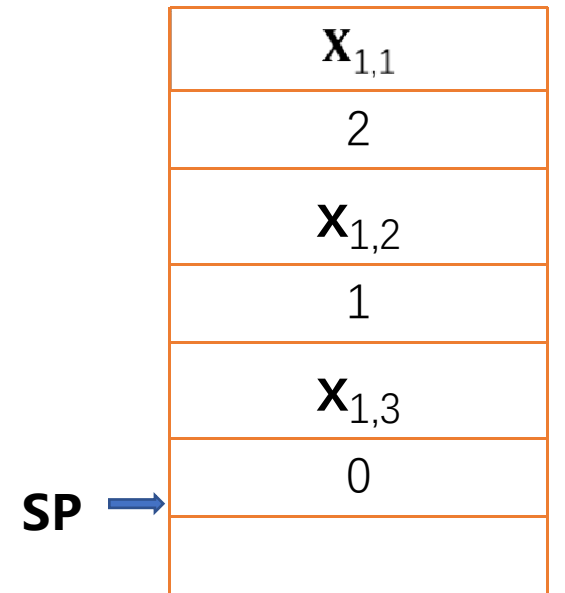
注: 每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码;

蓝色代表改变栈后继续执行的代码。

fact(0)返回结果给fact(1)

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd ra, 8(sp)    # 返回地址保存到栈中
      sd a0, 0(sp)    # 参数n=0保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1 # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1 # 若n-1 < 0, 返回值1并存入x10
      addi sp, sp, 16  # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra) # 返回fact(0)的调用者(执行Calc)
L1:   addi a0, a0, -1  # n = n - 1
      jal ra, fact    # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0 # fact(0)存到t1
      ld a0, 0(sp)    # 恢复调用者fact(1)的参数1
      ld ra, 8(sp)    # 恢复调用者的返回地址Calc
      addi sp, sp, 16 # 释放2个双字的栈空间
      mul a0, a0, t1  # 计算m * fact(m-1)
      jalr x0, 0(ra)  # 返回Calc
```



STEP 4

$X_{1,1}, X_{1,2}, X_{1,3} = ?$

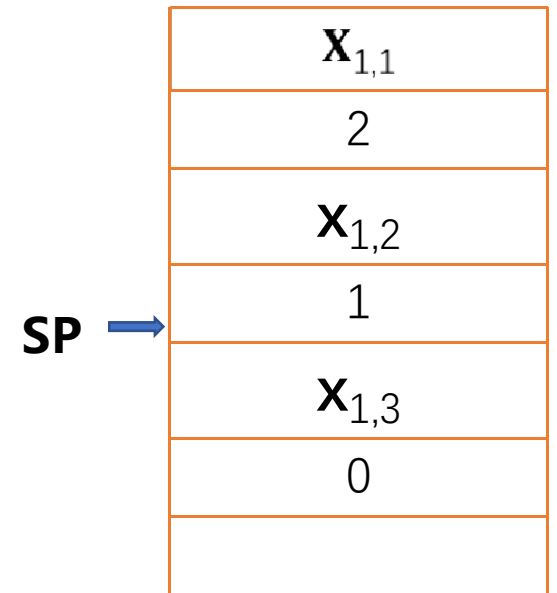
注: 每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码;

蓝色代表改变栈后继续执行的代码。

fact(1)返回结果给fact(2)

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd ra, 8(sp)    # 返回地址保存到栈中
      sd a0, 0(sp)    # 参数n=0保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1 # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1 # 若n-1 < 0, 返回值1并存入x10
      addi sp, sp, 16  # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra) # 返回fact(0)的调用者(执行Calc)
L1:   addi a0, a0, -1  # n = n - 1
      jal ra, fact    # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0   # fact(1)存到t1
      ld a0, 0(sp)    # 恢复调用者fact(2)的参数2
      ld ra, 8(sp)    # 恢复调用者的返回地址 (主程序)
      addi sp, sp, 16 # 释放2个双字的栈空间
      mul a0, a0, t1   # 计算1 * fact(1-1)
      jalr x0, 0(ra)  # 返回Calc
```



STEP 5

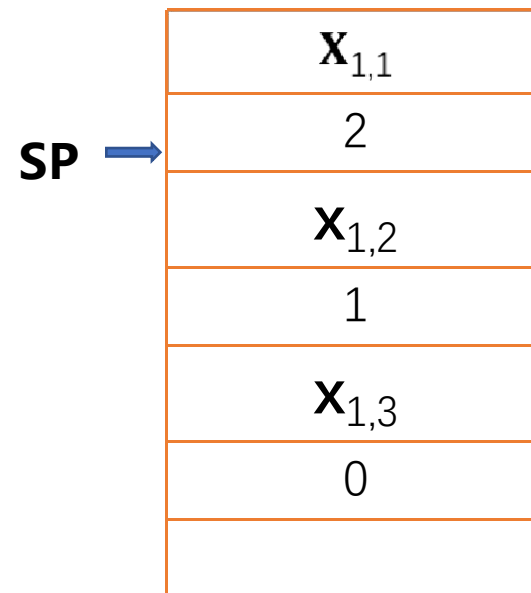
注：每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码；

蓝色代表改变栈后继续执行的代码。

计算出fact(2)

```
fact: addi sp, sp, -16 # 留出2个双字的栈空间
      sd ra, 8(sp)     # 返回地址保存到栈中
      sd a0, 0(sp)     # 参数n=0保存到栈中
      addi t0, a0, -1
      bge t0, zero, L1 # 若参数n-1 >= 0, 则跳转到L1
      addi a0, zero, 1 # 若n-1 < 0, 返回值1并存入x10
      addi sp, sp, 16  # 释放2个双字栈空间, 不用恢复
      jalr zero, 0(ra) # 返回fact(0)的调用者, Calc
L1:   addi a0, a0, -1   # n = n - 1
      jal ra, fact     # 调用fact(n-1), 重复直到fact(0).
Calc: addi t1, a0, 0    # 从fact(0)开始, 结果存到t1
      ld a0, 0(sp)     # 恢复调用者的参数1
      ld ra, 8(sp)     # 恢复调用者的返回地址Calc
      addi sp, sp, 16  # 释放2个双字的栈空间
      mul a0, a0, t1    # 计算2 * fact(2-1)
      jalr x0, 0(ra)   # 返回主程序
```



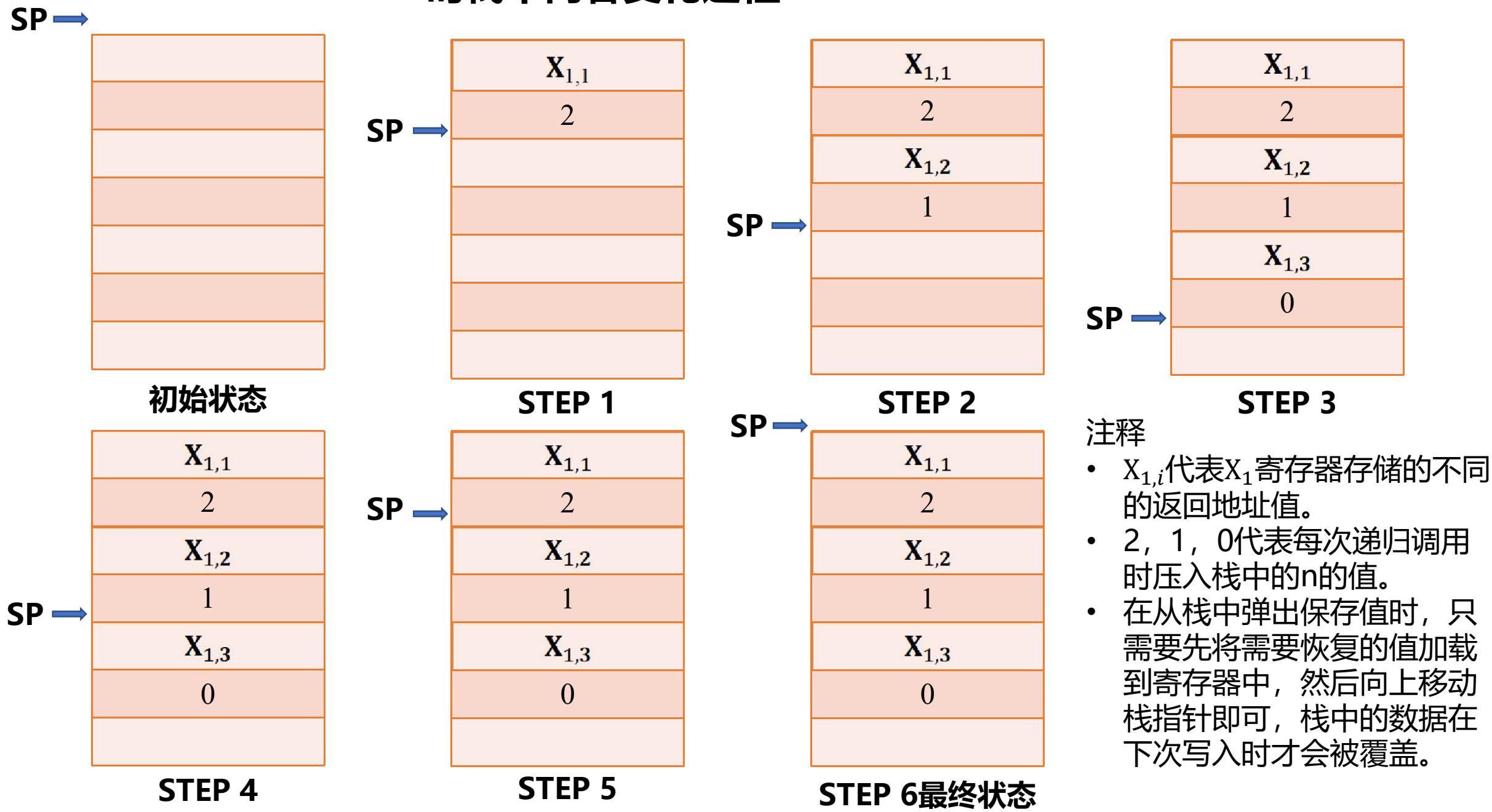
STEP 6

每一步都是从红色代码开始执行。

红色代表该步中改变栈的代码；

蓝色代表改变栈后继续执行的代码。

n=2时栈中内容变化过程



第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**

- 汇编语言简介
- RISC-V汇编指令概览
- RISC-V常用汇编指令
- 过程调用及栈的使用
- **模拟软件介绍**

RISC-V汇编语言规范

- 一个完整的 RISC-V 汇编程序多条语句（statement）组成。
- 一条典型的 RISC-V 汇编语句由3部分组成：
 - **label**（标号）：以冒号结尾的标识符，表示当前指令的位置。
 - **operation** 可以有以下几种类型：
 - instruction（指令）：对应二进制机器指令的字符串。例如addi、lw等。
 - pseudo-instruction（伪指令）：汇编器会生成多条实际指令(instructions)
 - directive(指示/伪操作): 通过类似指令的形式(以“.”开头)，通知汇编器如何控制代码的产生等，不对应具体的指令。常用的如 .data, .text...
 - macro: 自定义的宏，用 .macro/.endm 开始/结束。
 - **comment**（注释）：常用方式，“#”开始到当前行结束。

RISC-V汇编模拟软件

- RISC-V模拟软件很多： [官方网页](https://riscv.org/exchange/)(<https://riscv.org/exchange/>)
- 本课程使用RARS (RISC-V汇编程序和运行时**模拟器**)。
- RARS具有代码提示，模拟运行，调试，统计等功能。

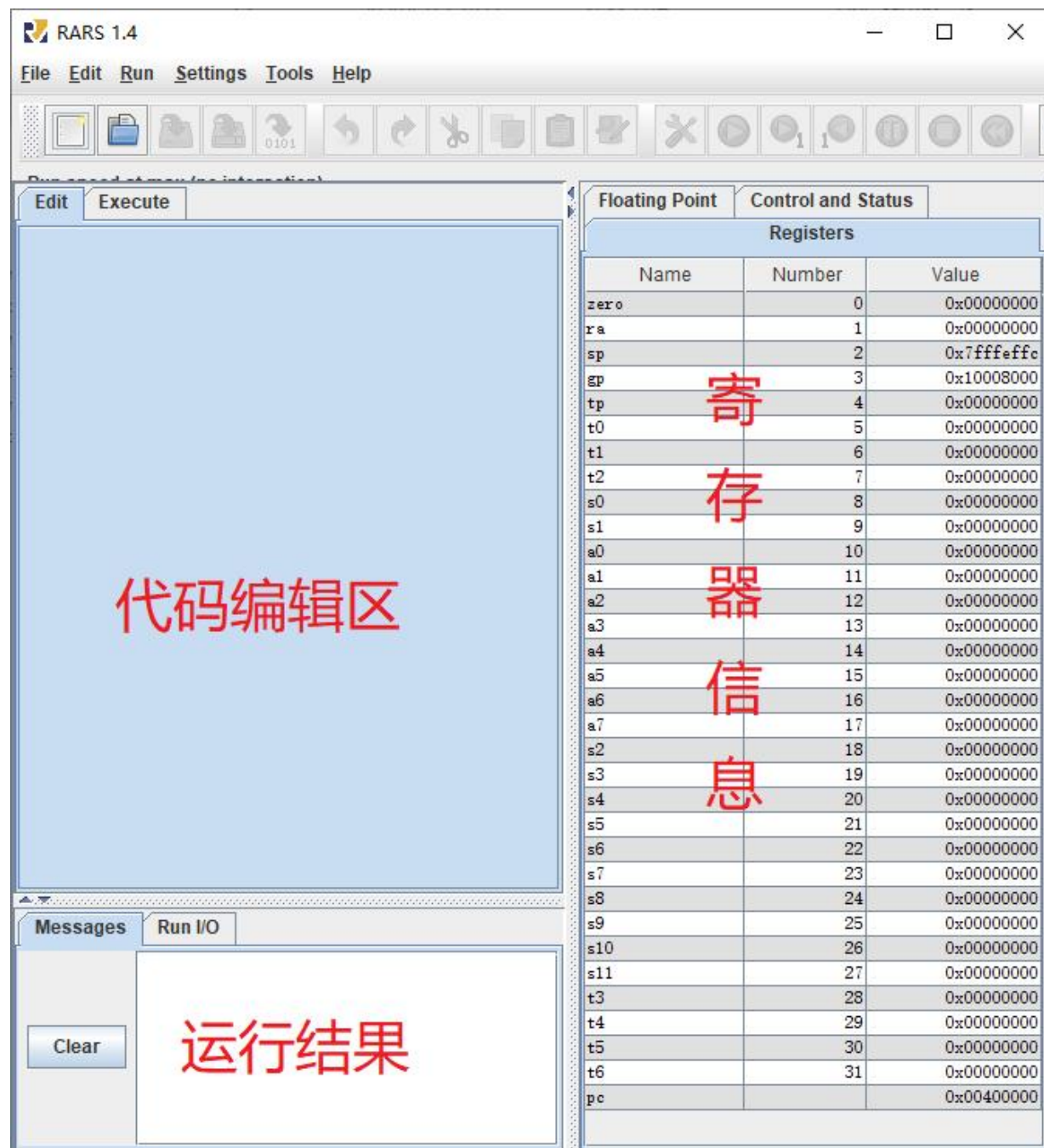


图1 RARS基本界面

RISC-V模拟软件

1) 运行方法

汇编代码编辑完成后，点击菜单栏的Run，选择Assemble即可进行汇编。也可以点击下图中用红色框框的图标进行汇编，如图2所示：

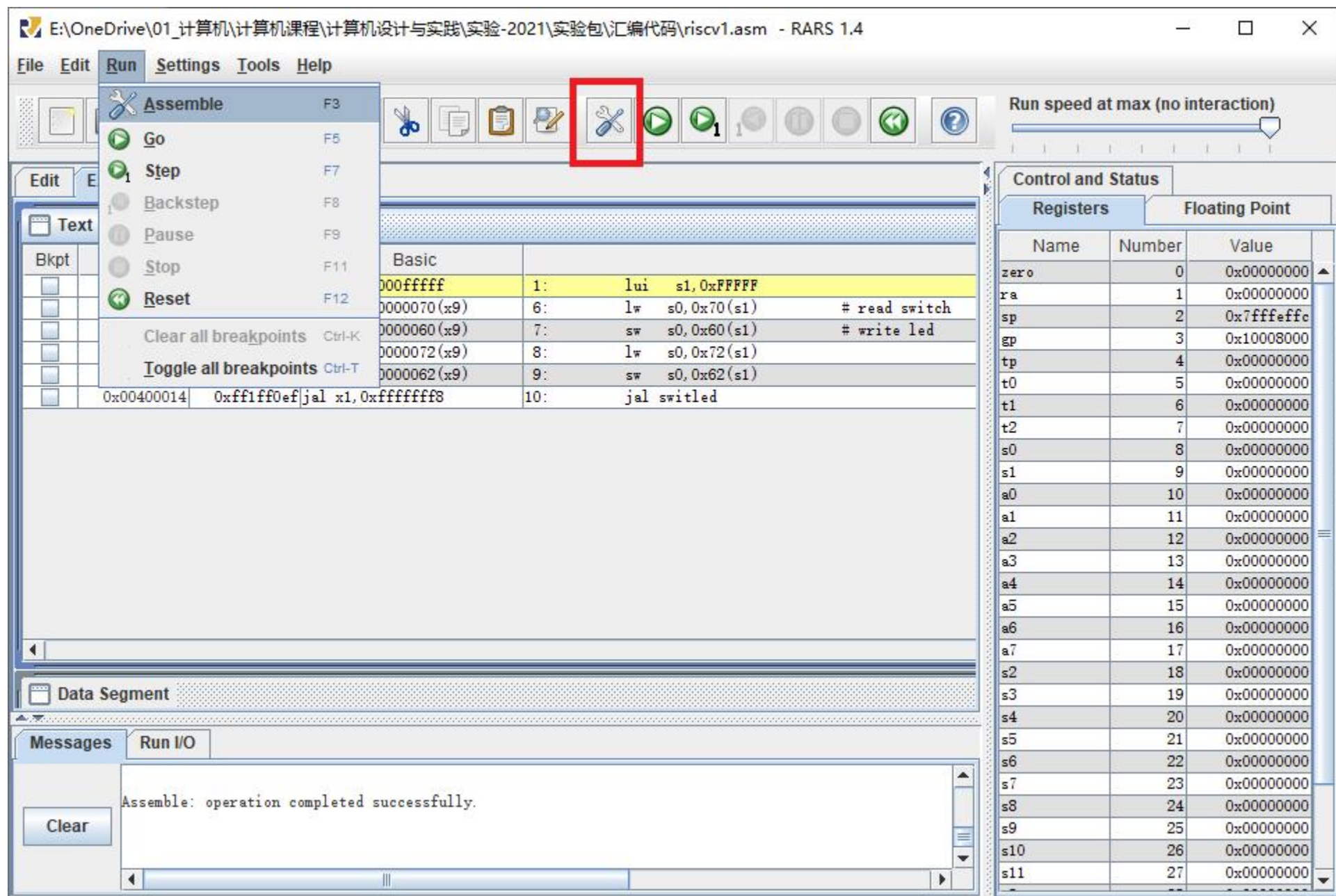


图2 RARS使用方法

例子：运行一个简单的数据交换swap

The image shows a RISC-V simulator interface. The main window displays assembly code for a swap function. The code is as follows:

```
1 addi x10, x0, 21
2 addi x11, x0, 20
3 jal x1, swap
4 j Done
5 swap:
6     add x6, x0, x10
7     add x10, x0, x11
8     add x11, x0, x6
9     jalr x0, 0(x1)
10 Done:
```

The simulator has a tabbed interface at the top with various assembly files. The 'Registers' tab is selected on the right, showing a table of registers and their values. The 'Messages' tab at the bottom shows the message: "program is finished running (dropped off bottom)".

Name	Number	Value
zero	0	0x0000000000000000
ra	1	0x0000000000004000
sp	2	0x000000007fffffc0
gp	3	0x0000000010008000
tp	4	0x0000000000000000
t0	5	0x0000000000000000
t1	6	0x0000000000000015
t2	7	0x0000000000000000
s0	8	0x0000000000000000
s1	9	0x0000000000000000
a0	10	0x0000000000000014
a1	11	0x0000000000000015
a2	12	0x0000000000000000
a3	13	0x0000000000000000
a4	14	0x0000000000000000
a5	15	0x0000000000000000
a6	16	0x0000000000000000
a7	17	0x0000000000000000
s2	18	0x0000000000000000
s3	19	0x0000000000000000
s4	20	0x0000000000000000
s5	21	0x0000000000000000
s6	22	0x0000000000000000
s7	23	0x0000000000000000
s8	24	0x0000000000000000
s9	25	0x0000000000000000
s10	26	0x0000000000000000
s11	27	0x0000000000000000
t3	28	0x0000000000000000
t4	29	0x0000000000000000
t5	30	0x0000000000000000
t6	31	0x0000000000000000
pc		0x0000000000004002

Line: 5 Column: 6 ☒ Show Line Numbers

Messages Run I/O

— program is finished running (dropped off bottom) —

Clear

运行一个简单的数据交换例子

EditExecute

Text Segment

Bkpt	Address	Code	Basic	Source
☐	0x00400000	0x01500513	addi x10,x0,21	1: addi x10, x0, 21
☐	0x00400004	0x01400593	addi x11,x0,20	2: addi x11, x0, 20
☐	0x00400008	0x004000ef	jal x1,0x00000004	3: jal x1, swap
☐	0x0040000c	0x00a00333	add x6,x0,x10	6: add x6, x0, x10
☐	0x00400010	0x00b00533	add x10,x0,x11	7: add x10, x0, x11
☐	0x00400014	0x006005b3	add x11,x0,x6	8: add x11, x0, x6
☐	0x00400018	0x00008067	jalr x0,x1,0	9: jalr x0,0(x1) # 返回调用函数

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data)

☒ Hexadecimal Addresses☒ Hexadecimal Values☐ ASCII

MessagesRun I/O

Reset: reset completed.

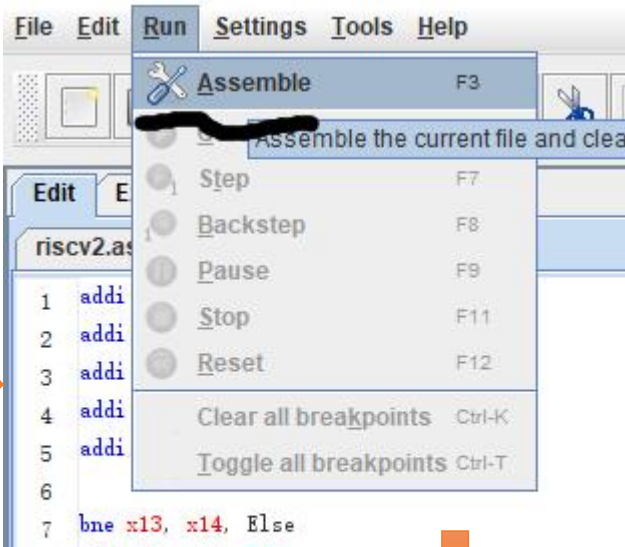
Clear

RegistersFloating PointControl and Status

Name	Number	Value
zero	0	0x00000000
ra	1	0x0040000c
sp	2	0x7ffffeffc
gp	3	0x10008000
tp	4	0x00000000
t0	5	0x00000000
t1	6	0x00000015
t2	7	0x00000000
s0	8	0x00000000
s1	9	0x00000000
a0	10	0x00000014
a1	11	0x00000015
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x00000000
s2	18	0x00000000
s3	19	0x00000000
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400018



```
1 addi x10,x0,3
2 addi x11,x0,5
3 addi x12,x0,7
4 addi x13,x0,8
5 addi x14,x0,9
6
7 bne x13, x14, Else
8 add x10, x11, x12
9 j Exit
10 Else: sub x10, x11, x12
11 Exit:
```



Text Segment				
Bkpt	Address	Code	Basic	
☐	0x00400000	0x00300513	addi x10,x0,3	1: addi x10,x0,3
☐	0x00400004	0x00500593	addi x11,x0,5	2: addi x11,x0,5
☐	0x00400008	0x00700613	addi x12,x0,7	3: addi x12,x0,7
☐	0x0040000c	0x00800693	addi x13,x0,8	4: addi x13,x0,8
☐	0x00400010	0x00900713	addi x14,x0,9	5: addi x14,x0,9
☐	0x00400014	0x00e69663	bne x13,x14,0x0000000c	7: bne x13, x14, Else
☐	0x00400018	0x00c58533	add x10,x11,x12	8: add x10, x11, x12
☐	0x0040001c	0x0080006f	jal x0,0x00000008	9: j Exit
☐	0x00400020	0x40c58533	sub x10,x11,x12	10: Else: sub x10, x11, x12

Floating Point		Control and Status	
Registers			
Name	Number	Value	
zero	0	0x00000000	
ra	1	0x00000000	
sp	2	0x7ffffeffc	
gp	3	0x10008000	
tp	4	0x00000000	
t0	5	0x00000000	
t1	6	0x00000000	
t2	7	0x00000000	
s0	8	0x00000000	
s1	9	0x00000000	
a0	10	0xffffffffe	
a1	11	0x00000008	
a2	12	0x00000007	
a3	13	0x00000008	
a4	14	0x00000009	
a5	15	0x00000000	
a6	16	0x00000000	
a7	17	0x00000000	
s2	18	0x00000000	
s3	19	0x00000000	
s4	20	0x00000000	
s5	21	0x00000000	
s6	22	0x00000000	
s7	23	0x00000000	
s8	24	0x00000000	
s9	25	0x00000000	
s10	26	0x00000000	
s11	27	0x00000000	
t3	28	0x00000000	
t4	29	0x00000000	
t5	30	0x00000000	
t6	31	0x00000000	
pc		0x00400028	

第三章 RISC-V汇编及其指令系统

- **RISC-V汇编语言**

- 汇编语言简介
- RISC-V汇编指令概览
- RISC-V常用汇编指令
- 过程调用及栈的使用
- 模拟软件RARS介绍

